`sqisign-selkie`: Implementing SQIsign v2.0.1 NIST-I in Rust

Deirdre Connolly

Selkie Cryptography, USA

Abstract. SQIsign has the smallest signatures and public keys of any post-quantum signature scheme. It is also one of the hardest to implement correctly. We describe `sqisign-selkie`, a Rust implementation of SQIsign v2.0.1 for NIST-I, built from the specification and C reference implementation. Along the way we found nine bugs traceable to specification ambiguities or implicit C reference conventions, and four classes of bugs arising from fixed-precision arithmetic. We catalog these, explain how they arose, and extract concrete lessons for anyone implementing isogeny-based schemes from a spec. We show how Rust's type system enforces cryptographic invariants at compile time, survey the published constant-time techniques for SQIsign's quaternion arithmetic, and describe a negative test vector suite (in C2SP/wycheproof JSON format) that detects vulnerabilities—not just bugs—in SQIsign verification, signing, and key generation.

Keywords: SQIsign · isogeny-based cryptography · Rust · constant-time · post-quantum signatures

1 Introduction

SQIsign [**?**] is the only isogeny-based signature scheme in Round 2 of NIST's additional signature standardization process. Its 148-byte signatures and 65-byte public keys at NIST-I are smaller than any other post-quantum scheme under consideration. But that compactness comes at a cost: implementing SQIsign requires quaternion algebras, the Deuring correspondence, dimension-2 theta-coordinate isogenies, and a signing algorithm that touches all of them.

The specification [**?**] is mathematically precise but leaves many implementation choices unstated. The C reference implementation fills in those choices—but they are embedded in code, not documented. An implementor working from the spec alone will make wrong guesses. We know because we did.

This paper documents `sqisign-selkie`, a Rust library for the NIST-I parameter set ($p = 5 \cdot 2^{248} - 1$) only. By fixing the parameter set at compile time, we specialize aggressively: field arithmetic uses a radix-51 Montgomery representation tuned to this prime, integer widths match proven worst-case bounds for NIST-I quaternion intermediates [**?**], and isogeny degree types are sized to the 246-bit maximum that NIST-I permits. We do not attempt to be generic over parameter sets.

We target four goals:

1. **Interoperability.** Verify and produce signatures that the C reference implementation accepts, byte-for-byte, against the NIST KAT vectors.

E-mail: durumcrustulum@gmail.com (Deirdre Connolly)

2. **Misuse resistance.** If a value is always positive and odd, it should be impossible to construct one that isn't. If a lattice operation requires Hermite Normal Form, the compiler should reject non-HNF input. This protects not just end-users of the public API, but us writing the internals, and anyone who maintains this code after us.

3. **Constant-time signing (goal, not yet achieved).** The specification does not discuss side channels. The C reference is variable-time for the entire quaternion layer. We traced the signing algorithm and confirmed that the quaternion operations compute over secret-key-derived data. Published attacks [**?**] demonstrate practical SPA on Cornacchia's algorithm. Constant-time execution is not a nice-to-have; it is a requirement. Our signing and key generation code is currently variable-time, with every secret-touching code path marked `TODO(ct)`. We plan to harden after achieving end-to-end interoperability (§7).

4. **Idiomatic Rust.** Methods on types over free functions. Invariants via distinct types, not runtime checks. Code that a Rust developer would recognize as natural [**?**].

This paper is a living document, updated as the implementation progresses.

Contributions.

- A catalog of thirteen implementation bugs (§4, §5) with root causes and lessons.
- Compile-time invariant enforcement via Rust's type system (§6).
- A survey of constant-time SQIsign techniques and a concrete hardening plan (§7).
- A negative test vector suite for SQIsign verification, signing, and key generation (§9)—portable JSON vectors in C2SP/wycheproof format that other implementations can consume to detect vulnerabilities including exponent overflow, non-canonical field elements, degenerate kernels, and cross-field splicing attacks.

2 Background

We assume familiarity with elliptic curves and isogenies at the level of the SQIsign specification [**?**], but not with the internals of theta-coordinate arithmetic or quaternion ideal algorithms. Where the spec is unclear, we say so; where the C reference makes an undocumented choice, we document it here.

3 Architecture

The implementation is organized into five layers:

`fields/` Prime field $\mathbb{F}_p$ (radix-51 Montgomery form) and quadratic extension $\mathbb{F}_{p^2}$.

`curves/` Montgomery curves, x-only projective arithmetic, torsion bases, 2^e-isogeny chains, and the `IsogenyDegree` newtype.

`surfaces/` Abelian surfaces in theta coordinates, $(2,2)$-isogeny chains (Algorithm 8.47), gluing, and splitting.

`quaternions/` Quaternion algebra $B_{p,\infty}$, lattices, ideals, HNF, L2 reduction, and `RepresentInteger`.

`deuring/` The Deuring correspondence bridge: `IdealToIsogeny`, `FixedDegreeIsogeny`, `SuitableIdeals`, action matrices.

3.1 Randomness and the `_derand` entry points

The SQIsign specification (Chapter 8) mandates **AES256-CTR-DRBG** (NIST SP 800-90A, §10.2.1) as the pseudo-random number generator for optimized implementations. The C reference follows this verbatim: its NIST PQC test harness (`PQCgenKAT_sign.c`) instantiates CTR_DRBG via `randombytes_init` with a 48-byte `entropy_input` and then services every call to `randombytes` from that state. Each KAT record in the `.rsp` files begins `seed = <96 hex chars>` – that 48-byte seed is the CTR_DRBG entropy input.

The 48 bytes are not arbitrary: for AES-256 the DRBG `seedlen` equals keylen + blocklen = 32+16 = 48. A shorter seed would leave part of the initial CTR_DRBG_Update state at zero and diverge from the NIST harness.

We expose two parallel APIs for keygen and signing:

- `SigningKey::generate<R: CryptoRngCore>(rng)` and `SigningKey::sign<R>(msg, rng)` sample 48 bytes of entropy from the caller's RNG and delegate to the derandomized entry points below. Production callers pass `OsRng`.

- `SigningKey::generate_derand(randomness: &[u8; 48])` and `SigningKey::sign_derand(msg, randomness)` take the 48-byte seed directly, instantiate a local `Aes256CtrDrbg` from it, and drive all internal sampling from that DRBG. These are the entry points that consume KAT seeds byte-for-byte.

Our `drbg` module implements AES256-CTR-DRBG directly on the RustCrypto `aes` crate, which provides AES-NI intrinsics on `x86_64` and ARMv8 cryptographic-extension intrinsics on `aarch64` out of the box. The module is a direct translation of the NIST reference (`CTR_DRBG_Instantiate`, `CTR_DRBG_Update`, `CTR_DRBG_Generate`): no derivation function, no reseed counter, no additional input. The DRBG is exposed as an `rand_core::RngCore`, so all generic sampling code in `quaternions::lattice` (rejection sampling, `rand_interval` inside `reduce_to_prime_norm`, `sample_from_ball`) drives it without further glue.

4 Bugs from Specification Ambiguity

During KAT verification testing against the C reference implementation, we discovered multiple bugs in our theta-coordinate $(2, 2)$-isogeny chain. Each bug originated from an ambiguity in the specification or an implicit convention in the reference code. We catalog them here with their root causes.

4.1 ProductToTheta coordinate convention

The spec describes the product theta structure abstractly via level-2 theta functions. The relationship between Montgomery projective coordinates $(X : Z)$ and the product theta representation is not stated explicitly. Our initial implementation used $(X - Z, X + Z)$ as "theta-like" coordinates—natural for Montgomery differential addition—but the C reference's `base_change()` uses raw (X, Z) directly:

```
fp2_mul(&null.x, &T->P1.x, &T->P2.x); // X1 * X2
fp2_mul(&null.y, &T->P1.x, &T->P2.z); // X1 * Z2
fp2_mul(&null.z, &T->P2.x, &T->P1.z); // Z1 * X2
fp2_mul(&null.t, &T->P1.z, &T->P2.z); // Z1 * Z2
```

Advice for implementers: When the spec is abstract about coordinate representations, the C reference is the ground truth.

4.2 Squared theta means square then Hadamard

In the SQIsign context, "squared theta" means component-wise squaring *followed by a Hadamard transform*. The C reference wraps this in `to_squared_theta()`, whose name does not reveal the Hadamard. Our generic isogeny evaluation (Algorithm 8.34) computed $\mathcal{H}(\text{inv} \cdot P^2)$ instead of $\mathcal{H}(\text{inv} \cdot \mathcal{H}(P^2))$.
Advice for implementers: Read the body of helper functions in the C reference, not just their names. "Squared theta" is not just squaring.

4.3 Cross-product index errors in GluingEval

Algorithm 8.39 computes `ADDComponents` on each curve component, producing (u_1, v_1, w_1) from curve 1 and (u_2, v_2, w_2) from curve 2. The U vector's second entry should be $u_1 \cdot w_2$ (cross-curve), but we wrote $u_1 \cdot w_1$ (same-curve).
Advice for implementers: In product-curve formulas, every multiplication involves one factor from each curve. Two factors with the same subscript are almost certainly wrong.

4.4 Projective inverse vs. field inversion

The gluing codomain needs the "inverse" of the dual theta null point $(\alpha, \beta, \gamma, 0)$. In projective coordinates this is computed via cross-products—pure multiplications, no field inversion. Our code called `Fp2::invert()`, which is both wrong and expensive.
Advice for implementers: In projective or theta coordinates, "inverse" almost always means projective inverse via cross-products. Actual field inversion is a red flag.

4.5 Reusing generic types for special patterns

The image of the kernel generator under gluing has the pattern $(x : x : y : y)$. Storing this in a generic 4-coordinate `JacobianPoint` creates ambiguity: `J.y` equals x (the second copy), not y. The scaling step used `J.x` and `J.y` for the two distinct values, but both hold x.
Advice for implementers: Do not reuse a generic n-coordinate type to store a value with a special pattern. Use a dedicated type, or at minimum document which fields hold which logical values.

4.6 Torsion basis swap convention

The C reference's `ec_curve_to_basis_2f_from_hint()` deliberately stores $P - Q$ in `B.Q` and Q in `B.PmQ`:

```
difference_point(&PQ2->Q, &P, &Q, curve);   // B.Q = P - Q
copy_point(&PQ2->P, &P);                    // B.P = P
copy_point(&PQ2->PmQ, &Q);                  // B.PmQ = Q
```

This means the three-point ladder computes $P + [m](P - Q)$, not $P + [m]Q$. The swap ensures the multiplied point avoids $(0, 0)$ for differential addition. This is not documented in the spec.

The convention is load-bearing through M_{chl}. The shuffled layout is not local to TorsionBasisFromHint. Every site that consumes a basis through LadderBiscalar — including signing's SetChangeOfBasisMatrix (Algorithm 4.8) and verify's $(P, Q) \leftarrow M_{\text{chl}} \cdot (P_{\text{can}}, Q_{\text{can}})$ (Algorithm 4.9, line 14) — reads the triple as $(P, P-Q, Q)$. The matrix M_{chl} is therefore encoded in the $(P, P-Q)$ frame, not in the spec's nominal (P, Q) frame.

If signing builds the bases that feed SetChangeOfBasisMatrix with the naïve $(P, Q, P-Q)$ layout, the byte-level signature parses, verify runs, and the matrix application produces a singular pair on the codomain — in our debugging both rows of the resulting basis on E_{chl} ended up as the same point on the curve, bit-for-bit. The $(2,2)$-chain in Algorithm 4.9 either crashes inside lift_basis (the lift expects three non-degenerate projective representatives) or completes with $\#\{i, j : U_{i,j}(0) = 0\} = 0$, and verify rejects. Without intermediate diagnostics it looks like "the chain is just wrong" rather than a basis-layout mismatch.

We hit this when end-to-end signing first started producing a signature: the verify-side P_{chl} and Q_{chl} differed only at the layout level. Switching the signing-side `basis_aux` and `basis_chl` construction from `from_propagated(P, Q, PmQ)` to `from_propagated(P, PmQ, Q)` aligned the convention and let verify accept our own signatures.

Advice for implementers: Undocumented conventions in reference code can look like bugs in your own code. Verify against the reference before "fixing" something that isn't broken — and in particular, check that the convention is uniform from TorsionBasisFromHint all the way through the M_{chl} encoding and decoding pipeline.

4.7 Matrix storage transpose in ChangeOfBasis

Algorithm 2.5 (ChangeOfBasis) returns four scalars x_1, x_2, x_3, x_4 satisfying

$$Q_1 = [x_1]P_1 + [x_2]P_2, \qquad Q_2 = [x_3]P_1 + [x_4]P_2,$$

and packs them into a 2×2 matrix. The spec does not say which of two natural layouts is on the wire:

$$M_{\text{row}} = \begin{pmatrix} x_1 & x_2 \\ x_3 & x_4 \end{pmatrix} \quad \text{vs.} \quad M_{\text{col}} = \begin{pmatrix} x_1 & x_3 \\ x_2 & x_4 \end{pmatrix}.$$

The C reference uses M_{col}. `change_of_basis_matrix_tate` stores `mat[0][0]=x1, mat[1][0]=x2, mat[0][1]=x3, mat[1][1]=x4`, so column 0 is the coordinates of Q_1 in the (P_1, P_2) basis and column 1 is the coordinates of Q_2. `matrix_application_even_basis` consumes this layout by columns: `bas→P = [mat[0][0]] P + [mat[1][0]] Q` (column 0 yields the new first basis point).

We initially stored the matrix in M_{row} form. This was not detectable from KAT verification alone — KAT signatures arrive on the wire already in the C-ref layout, our `from_bytes` deposits the four scalars into the same four positions, and our `mul` reads columns just like the C ref. So consuming externally-produced matrices worked.

The defect manifested only when our own signing code produced M_{chl}. `from_bases` writes the row-major convention, `to_bytes` serialises the entries in positional order, and the resulting wire bytes encode M_{chl}^T rather than M_{chl}. Verify on our own signatures (or any other implementation reading the wire) then applies the transposed matrix to the canonical basis on the challenge codomain. The points it gets are not garbage — they are $([x_1]P + [x_3]Q, [x_2]P + [x_4]Q)$, i.e., the application of M^T — so the $(2,2)$-chain proceeds, the curve arithmetic does not crash, and verify rejects with no useful signal.

The same bug exists in M_{sk} inside Algorithm 4.1 (KeyGen) because key generation goes through the same ChangeOfBasis call. KAT secret keys parse correctly because we read them by position; new keys we generate ourselves would carry transposed M_{sk} and refuse to round-trip through any spec-faithful verifier.

Catching this earlier. A four-line unit test `from_bases_mul_roundtrip` — "construct a target as $M \cdot$ source for a known M with small entries, run `from_bases` to recover M', check that `mul`(M', source) equals the target" — catches it instantly, without going anywhere near the response phase. We did not have this test. Sign + verify is a 600-second feedback loop; the roundtrip test runs in milliseconds and would have isolated the defect to one function.

Advice for implementers. When the spec specifies a matrix as four scalars and a row of pseudocode, write a regression test that fixes a known matrix, runs your basis construction + matrix recovery + matrix application, and asserts equality. Otherwise the only signal you get is from a slow, end-to-end test that fails for many reasons at once, and it takes a careful read of both `change_of_basis_matrix_tate` and `matrix_application_even_basis` in the C reference to notice that one stores by columns and the other reads by columns.

4.8 Spec's 2^{f-e} matrix scaling is wrong for reduced bases

Algorithm 2.5 step 4 specifies that the returned matrix entries are

$$x_i \leftarrow 2^{f-e} \cdot \log_\zeta(\zeta_i),$$

i.e., the e-bit discrete log scaled up to a 2^f-precision value. The intent is that the matrix can be applied directly to a full 2^f-torsion basis: applying $x_i \in [0, 2^f)$ via biscalar multiplication recovers the corresponding 2^e-torsion linear combination after the inherent 2^{f-e} scaling of the basis.

In practice (and in the C reference), Algorithm 4.8 reduces both bases to order 2^e via doublings *before* computing the matrix. Once the bases are reduced, the 2^{f-e} scaling is wrong: applying an f-bit-shifted entry to a 2^e-torsion point produces a value whose useful bits live above the wire encoding's window.

The wire encoding stores $\lceil e/8 \rceil$ bytes per entry (§4.6). After the spec's shift, those low $\lceil e/8 \rceil$ bytes are mostly zero ($f - e$ bits of leading zero from the shift), with only a few bits of the actual dlog spilling into the encoded range. Verify reads near-zero entries, applies them as scalars, and gets a near-zero output basis. The downstream $(2, 2)$-chain then fails with a fully-degenerate theta null (`splitting: zeros=10`).

The C reference's `_change_of_basis_matrix_tate` omits the spec's 2^{f-e} scaling and stores the raw dlog values. This is undocumented but necessary for the wire format to work.

Advice for implementers. Read Algorithm 4.8, observe the basis reduction, and treat Algorithm 2.5's step 4 scaling as informational — applicable only if you skip the reduction. If you follow the spec literally, your matrix entries are silently zero on the wire and verification fails with no useful diagnostic (the chain reaches its terminal step with all-zero theta null). A unit test that fixes a known reduced-basis pair and asserts the round-trip catches this in milliseconds; the end-to-end test takes ten minutes and fails for many reasons at once.

4.9 Matrix exponent missing `HD_extra_torsion`

Algorithm 4.8 computes the change-of-basis matrix at exponent e where the bases have been pre-reduced to order 2^e. The exact value of e matters: dlog precision is e bits, so an under-specified e silently truncates the high bits of every matrix entry.

We initially used $e = e'_{\text{rsp}} + r_{\text{rsp}}$ (line 13 of Algorithm 4.8's scaling step in the spec), missing the `HD_extra_torsion` $= 2$ bits required by the dim-2 chain that follows in Algorithm 4.9. The C reference uses $e =$ `pow_dim2_deg_resp` + `HD_extra_torsion` + `two_resp_length` $= e'_{\text{rsp}} + r_{\text{rsp}} + 2$ for both basis reduction and matrix computation, so its dlog has the full precision the chain needs.

The spec's Algorithm 4.8 is silent on this +2. The bases that arrive at Algorithm 4.8 (from Algorithm 4.5, SplitAuxiliaryIsogeny) do carry the extra two bits; the spec just doesn't tell you to preserve them through the matrix. Without preservation, verify applies the matrix and gets a basis that's correct mod $2^{e'_{\text{rsp}}+r_{\text{rsp}}}$ but wrong in the high two bits — the same two bits the chain will consume for `HD_extra_torsion`. Symptoms: chain runs for a few steps then collapses to all-zero theta nulls.

Advice for implementers. When the spec gives an exponent for matrix arithmetic, cross-reference what bits the downstream consumer needs. If Algorithm 4.9's $(2,2)$-chain needs $2^{e'_{\text{rsp}}+2}$-torsion, the matrix that applies to its input basis must also carry those two bits. The C reference's `reduced_order` expression is the source of truth.

4.10 32-bit truncation in dlog recursion

NormalizedDlog (Algorithm 2.4) reduces the dlog computation modulo 2^e by splitting the exponent: it recurses on the upper half $e' = \lceil e/2 \rceil$ to recover the low bits k', then computes $\zeta_1/\zeta_0^{k'}$ to reduce the residual problem to the upper half. Implemented naively in Rust, the $\zeta_0^{k'}$ exponentiation is the only sub-step that takes a *large* exponent — up to $2^{e'}$.

We initially passed k' through a `u32` cast:

```
let z1_double_prime = target / &self.pow(k_prime.as_limbs()[0] as
    u32);
```

A comment justified this with "$k' < 2^{e'}$ which fits in `u32` for $e' \leq 124$," which is straightforwardly wrong: 2^{124} does not fit in a `u32`; only 2^{32} does.

In Algorithm 4.8 the matrix entries are dlogs of cross-pairings at exponent $e'_{\text{rsp}}+r_{\text{rsp}}+2 \approx 126$. At $e \approx 126$ the recursion's k' values regularly exceed 2^{32}. The cast then silently truncates, $\zeta_0^{k'}$ is computed for the wrong exponent, and the final dlog is junk.

Symptoms. The bug only fires for "large enough" inputs. KAT round-trips, the existing `dlog_round_trip_large` test (which used ζ^{42}), and any dlog test with $k < 2^{32}$ all pass cleanly. The first real exposure is in M_{chl} at exponent ~ 126: roughly half the cross-pairings have their dlog truncated to a fixed root of unity, and several entries in the resulting matrix collapse to the same value (most strikingly, every entry's low 64 bits are `0xffff_ffff_ffff_ffff` when the dlog returns $-1 \bmod 2^e$). Verify reads the matrix, applies it to the canonical basis, and gets $P_{\text{chl}} = Q_{\text{chl}}$ since both columns now have identical scalar combinations. The $(2,2)$-chain immediately fails with a fully-degenerate theta null.

Fix. Use a scalar-sized exponent throughout:

```
let z1_double_prime = target / &self.pow_scalar(&k_prime);
```

`pow_scalar` walks all 256 bits of the `Scalar`, so it handles dlogs up to the full torsion exponent $f = 248$.

Catching this earlier. A regression test `dlog_round_trip_above_u32` that exercises a known $k > 2^{32}$ catches the bug in milliseconds. We did not have one; the previous large-dlog test used $k = 42$. A type-system signal ("the exponent is a `Scalar`") would have prevented the `u32` cast from being written in the first place.

Advice for implementers. When recursing on a 2-power dlog, the recursive sub-call's output is a full e'-bit value, not a small index. Test with exponents $> 2^{32}$ and $> 2^{64}$ explicitly — small-value tests will not exercise the truncation path, and the truncation only manifests downstream as "the matrix makes the basis collapse," which is a long way from the actual defect site.

4.11 Cubical Tate third-arg sign convention in `from_bases`

The cubical Tate pairing $t_{2^e}(P, Q, P{+}Q)$ takes a third projective input that fixes the sign branch of the differential ladder. Passing $P{+}Q$ versus $P{-}Q$ as the third argument produces values that are inverses of each other: $t_{2^e}(P, Q, P{-}Q) = t_{2^e}(P, Q, P{+}Q)^{-1}$. Both

satisfy bilinearity in their respective conventions, both pass "output is a 2^e-th root of unity" tests, and the two conventions are individually self-consistent — but mixing them silently inverts the dlog.

Algorithm 2.5 step 1 sets $\zeta \leftarrow t_{2^e}(P_1, P_2)$ without naming the third argument's sign. Our `from_bases` initially passed `full_basis.RS` (which is $P_1 - P_2$ in the shuffled basis layout — §4.6) for the base ζ, while the four cross-pairings ζ_i used target+full (the SUM, computed via Jacobian addition). The base ζ then equals T^{-1} where the cross-pairings equal T_i, so $\log_\zeta(\zeta_i)$ returns $-x_i \bmod 2^e$ instead of x_i.

Symptom. For a target basis with all coefficients in $\{1, 2\}$ (a fairly common case in Algorithm 4.8), every dlog comes out near $2^e - 1$ or $2^e - 2$ — every cell of the resulting matrix has its low limbs entirely set to `0xff...ff`, with at most a few low bits differing. Verify accepts the matrix syntactically (entries are within bounds), applies it to the canonical basis, and gets a *non-degenerate but wrong* basis; the $(2, 2)$-chain runs to completion, lands on a non-product theta null, and verification rejects.

Fix. Compute ζ using the same SUM convention as the cross-pairings: $\zeta \leftarrow t_{2^e}(P_1, P_2, P_1 + P_2)$ where $P_1 + P_2$ is computed via the same Jacobian `x_add_sub` that produces the cross-sums.

Catching this earlier. This bug bypasses every test that exercises only *one* of the two conventions. `tate_bilinear_in_first_arg_with_diff`, `tate_bilinear_in_first_arg_with_sum`, `tate_antisymmetric`, and the original `dlog_round_trip_large` all hold internally; what they miss is that `from_bases` mixes them. A unit test that constructs a known matrix with small positive entries and asserts `from_bases` recovers them with the right sign would have caught it.

Advice for implementers. When an algorithm uses both "$\zeta = t(P, Q)$" and "$\zeta_i = t(\text{target}, P_j)$," fix one third-argument convention everywhere. The sign-flip between SUM and DIFFERENCE conventions is invisible at every single call site — only the cross-call interaction reveals it.

4.12 Different formulas for the last two chain steps

The C reference uses three different formula variants across the $(2, 2)$-isogeny chain, controlled by two boolean flags `hadamard_bool_1` and `hadamard_bool_2`:

- **Normal steps** ($i < n - 2$): `bool_1=0, bool_2=1`. Codomain is Hadamard-transformed (standard form). Eval computes $\mathcal{H}(\text{precomp} \cdot \mathcal{H}(P^2))$.
- **Penultimate step** ($i = n - 2$): `bool_1=0, bool_2=0`. Codomain stays in dual form (no Hadamard). Eval computes $\text{precomp} \cdot \mathcal{H}(P^2)$.
- **Ultimate step** ($i = n - 1$): `bool_1=1, bool_2=0`. Input gets an extra Hadamard before squaring; codomain in dual form. Eval computes $\text{precomp} \cdot \mathcal{H}(\mathcal{H}(P)^2)$.

The splitting step expects its input in dual form—what `bool_2=0` produces. Our code used the normal-step formula (`bool_2=1`) for all steps, feeding the splitting a Hadamard-transformed null point that has no zero $U_{i,j}(0)$ entry. The chain ran correctly for 125 steps and produced nonsense at the end.

Advice for implementers: The last few steps of an isogeny chain often need special handling. The spec describes the generic step uniformly; the `hadamard_bool` optimization is an implementation-level choice in the C reference that avoids redundant Hadamard transforms at the boundary between the chain and the splitting step. Check the loop structure, not just the per-step formula.

4.13 `extra_torsion=false` chain tail: 4-iso / 2-iso formula transcription error

The C reference exposes a second chain mode for when the input kernel has order exactly 2^e rather than the 2^{e+2} expected by §4.12. Selected by `extra_torsion=false`, it replaces the 8-torsion penultimate and ultimate steps (`theta_isogeny_compute(...,0,0)` and `theta_isogeny_compute(...,1,0)`) with two dedicated lower-torsion isogenies:

- *Penultimate*: 4-isogeny via `theta_isogeny_compute_4(...,0,0)`, formulated for a single 4-torsion kernel point with a square-root recovery for the missing cross term.
- *Ultimate*: 2-isogeny via `theta_isogeny_compute_2(...,1,0)`, formulated from the domain null point alone, with three square roots producing the codomain components.

This tail is reached from `dim2id2iso_ideal_to_isogeny_clapotis` in the outer chain (`dim2id2iso.c:1183`), the only call site in the C reference that passes `extra_torsion=false`.

The spec (Algorithms 8.32, 8.33) and the C reference present these formulas in different projective representatives, and the two presentations differ in which of the two square roots multiplies which component. The C reference computes

$$\text{null}.x = \text{TT1}.x \cdot \text{TT2}.x \cdot \text{sqaacc},$$

while a literal reading of the spec leads to

$$\text{null}.x = \text{TT1}.x \cdot \text{sqaabb} \cdot \text{TT2}.x.$$

The swap is not a projective scalar: it takes the null point into a different projective class, and the splitting step downstream finds zero vanishing $U_{i,j}(0)$ entries instead of one. The 2-isogeny formula has a parallel discrepancy.

Diagnostic. Pure inspection didn't reveal the swap. Both formulas type-check, both produce non-zero output at every step, and the chain fails only at the final splitting check, far downstream of the actual mistake. Instrumenting the C reference's chain to emit per-step theta null points, mirroring the same instrumentation in our chain, driving identical kernel bytes through both, and diffing the two streams localizes the bug to the first chain step that uses an affected codomain formula. After the formula fix, the two streams agree byte-for-byte from the gluing through the input to the splitting step.

Splitting predicate strength. The C reference's `splitting_compute` accepts a final null point only when exactly one $U_{i,j}(0)$ vanishes *and*, on the `extra_torsion=true` path, that zero appears at the specific index 8 corresponding to the canonical product structure of protocol-constructed kernels. The two checks are independent: malformed inputs can produce a single zero at the wrong index, and a count-only predicate accepts where the index check rejects. We restored the index-8 discriminator on the `extra_torsion=true` path; the `extra_torsion=false` tail keeps the count-only check (matching the C reference's `zero_index` parameter of -1 at this call site).

Advice for implementers: When transcribing a formula that exists in two presentations (the spec and a reference implementation), pick one and follow it literally; do not blend. Cross-check with a byte-level diff against an instrumented reference rather than synthetic test kernels: not every isotropic subgroup of $E_1[2^e] \times E_2[2^e]$ is a valid chain input (§5.6), and the chain can run cleanly on an invalid kernel and reject only at the splitting step, leaving no way to distinguish "invalid input" from "buggy chain" from the final null alone. Make sure your splitting predicate is no weaker than the reference's: a count-only check that accepts where an index-discriminating check rejects will mask divergences exactly where they're hardest to chase down.

4.14 Outer chain needs two torsion bits the spec doesn't mention

This bug cost us *days* of debugging. It sits at the boundary between Algorithm 3.13 (IdealToIsogeny) and Algorithm 8.47 (Isogeny22ChainWithTorsion), and manifests as a chain that runs for 246 steps and then produces an abelian surface that is not a product—even when the input kernel is valid.

The setup. Algorithm 3.13 constructs a kernel on $E_u \times E_v$ of the form

$$T_1 = (\varphi_u(P_0),\, \theta \cdot \varphi_v(P_0)), \quad T_2 = (\varphi_u(Q_0),\, \theta \cdot \varphi_v(Q_0))$$

where φ_u, φ_v are the odd-degree isogenies produced by two calls to Algorithm 3.15, (P_0, Q_0) is the canonical 2^f-torsion basis of E_0, and θ is the quaternion constructed from β_1, β_2 in lines 5–6. Before passing (T_1, T_2) to the chain, the spec says "double as needed" to reduce from the 2^f-torsion on which $\varphi_u, \varphi_v, \theta$ were evaluated to the 2^e-torsion kernel of the outer isogeny. A natural reading is "double by $f - e$ times." That's wrong.

The chain convention. Algorithm 8.47 runs a chain of e 8-torsion $(2,2)$-isogenies and consumes a kernel of order 2^{e+2}—two bits larger than the resulting isogeny's degree. The extra bits are *internal*: the penultimate and ultimate steps fold the kernel's 4-torsion and 2-torsion residue into the last two $(2,2)$-isogenies via the `hadamard_bool` mechanism from §4.12. Without those bits, the last two steps double past identity, the resulting theta null collapses, and every $U_{i,j}(0)$ either vanishes (the fully degenerate case) or none of them do. Either way, the codomain is not a product and the signing key computed from it is wrong.

We originally doubled the kernel by $f - e$ steps, landing at a kernel of order exactly 2^e. Our $(2,2)$-chain then ran all the way to the end, the splitting code silently picked one of the ten $U_{i,j}(0)$ indices anyway, and keygen returned a bad curve. The bug reproduced on 99% of random seeds.

The fix. Double by $f - e - 2$ instead of $f - e$. The kernel enters the chain with order 2^{e+2}, the last two steps have the extra torsion bits they expect, and the chain terminates at a proper elliptic product. After applying the fix, `generate_runs` succeeds on its first outer-chain attempt in 10/10 consecutive runs and the terminal split count is exactly 1 every time.

The correction depends on $e \le f - 2$: if $e = f$ or $e = f - 1$, the 2^{e+2}-torsion doesn't exist on the target curve. For SQIsign we eliminate those cases in SuitableIdeals by requiring $v_2(\gcd(u, v)) \ge 2$ in find_uv_from_lists (§4.14), which leaves $e \le f - 2$. This filter passes $\approx 81\%$ of random (β_1, β_2) pairs in practice.

What the C reference does. The C reference implements two chain variants keyed on a boolean `extra_torsion` (`hd/ref/lvlx/theta_isogenies.c:1088`). With `extra_torsion = true`, it matches our implementation (kernel 2^{e+2}, pure 8-torsion chain). With `extra_torsion = false`, it runs a shorter 8-torsion chain followed by a dedicated 4-torsion step and a dedicated 2-torsion step, consuming a kernel of order 2^e. Algorithm 3.13 line 9 calls the outer chain with `extra_torsion = false` (`dim2id2iso.c:1128`); the inner Algorithm 3.15 calls it with `extra_torsion = true` (`dim2id2iso.c:181`). The spec mentions neither variant; an implementer reading the spec alone would find no indication that the outer chain is special in any way.

Advice for implementers. When you couple Algorithm 3.13 with Algorithm 8.47 for the first time, check that the kernel delivered to the chain has two bits of torsion above the "obvious" kernel order. If your chain matches the spec's uniform 8-torsion formulation, the padding must come from the caller. A 100%-failure-rate split is the visible symptom;

a 50%-failure-rate split means you have the kernel right but a separate bug. We flag this as a spec ambiguity in §??.

4.15 $\mathbb{F}_{p^2}$ square root canonicalization

This was the single most impactful bug we encountered, and we devote extra space to it because its consequences are subtle and far-reaching.

Every nonzero square in $\mathbb{F}_{p^2}$ has exactly two roots, r and $-r$. The SQIsign specification (Algorithm 8.12, the `difference_point` subroutine of `TorsionBasisFromHint`) says "compute a square root" without specifying which one. We initially returned whichever root the Tonelli–Shanks formula produced.

The C reference's `fp2_sqrt` (described in Aardal et al. [?], §3.1) has a five-line canonicalization step at the end: it negates the output whenever the real part is odd (as an integer in $[0, p)$), or when the real part is zero and the imaginary part is odd. This ensures a unique, deterministic "even" root for every input.

Without this canonicalization, `projective_difference` nondeterministically adds $+\sqrt{\Delta}$ or $-\sqrt{\Delta}$ to B_{XZ}, producing two affinely distinct candidates for $x(P - Q)$. Only one matches the C reference's `difference_point`. The wrong choice propagates through `ladder3pt` (producing a different kernel generator), the challenge isogeny (landing on a different curve E_{chl} with a different j-invariant), and every subsequent step. In our case, the challenge curve's j-invariant was `0x03007c`... instead of the correct `0x026558`...—completely unrelated values, not just a sign flip.

The key insight is that the choice of square root is *not absorbed by projective equivalence.* The formula $x_{P-Q} = (\sqrt{\Delta} + B_{XZ} \ : \ B_{ZZ})$ is not projectively equivalent to $(-\sqrt{\Delta} + B_{XZ} \ : \ B_{ZZ})$; these are genuinely different points. Any implementation of SQIsign that computes torsion bases via `difference_point` must canonicalize its $\mathbb{F}_{p^2}$ square root.

Advice for implementers: Any time a specification says "compute a square root" in $\mathbb{F}_{p^2}$, the implementation *must* specify and enforce a canonical choice. The even-real-part convention used by the C reference is simple and should be the default for any SQIsign implementation. This is not documented in the SQIsign specification as of v2.0.1.

4.16 Never recompute $P-Q$ via `DifferencePoint`

A torsion basis $(P, Q, P-Q)$ is produced once by `TorsionBasisFromHint`. The third component $P-Q$ is computed via `DifferencePoint`, which takes an $\mathbb{F}_{p^2}$ square root. We discovered—after fixing the sqrt canonicalization—that $P-Q$ must *never* be recomputed from P and Q in any subsequent operation. Even with a canonical sqrt, the two roots of `DifferencePoint` yield the affinely distinct points $x(P-Q)$ and $x(2P-Q)$, and there is no guarantee that a fresh call returns the same one.

We found three separate sites where recomputation caused failure:

1. **After scaling.** We doubled P and Q to reduce their order, then called `projective_difference` on the doubled points instead of doubling the original $P-Q$ alongside.

2. **In the matrix application.** $M_{\text{chl}} \cdot (P, Q)$ produced P' and Q' via two biladder calls. We computed $P'-Q'$ via `projective_difference` instead of a third biladder with scalars $(a-b, c-d)$, as the C reference does in `matrix_scalar_application_even_basis`.

3. **Before the $(2,2)$-chain.** After the even response isogeny, we recomputed $P-Q$ from the images instead of pushing the original $P-Q$ through the isogeny as a third evaluation point.

Each fix was individually necessary; all three together were sufficient to make KAT vector 0 verify.

Advice for implementers: In x-only Montgomery arithmetic, $P{-}Q$ is a *derived value* that cannot be reconstructed from $x(P)$ and $x(Q)$ alone (because $x(P{-}Q) = x(P{+}Q)$ on the Kummer line, and `DifferencePoint` resolves this ambiguity via a square root whose sign is fragile). Treat $P{-}Q$ as a first-class member of the basis triple and propagate it through every transformation.

4.17 Premature optimization of cubical pairing arithmetic

Algorithm 8.14 (CubicalDiffAdd) ends with line 7,

$$X_2 \leftarrow X_2 / x(P{-}Q),$$

which divides one coordinate of the output cubical point by the x-coordinate of the difference. An "optimization" that looks free moves the factor onto the other coordinate instead:

$$Z_2 \leftarrow Z_2 \cdot x(P{-}Q).$$

Both choices produce points with the same affine ratio X/Z, so on the Kummer line they represent the same point; the rewrite saves one $\mathbb{F}_{p^2}$ inversion per ladder step. Our implementation took the second form. Every test that probed only the affine projection passed: CubicalLadder produced points on the curve, `tate_pairing_is_root_of_unity` confirmed the final value lay in μ_{2^e}, and dlog round-trips recovered the right scalars.

The cubical pairing pipeline is not projectively invariant. CubicalTranslate (Alg. 8.16) is bilinear in (X, Z), not in X/Z:

$$X' \leftarrow X(T)\,X(P) - Z(T)\,Z(P), \quad Z' \leftarrow Z(T)\,X(P) - X(T)\,Z(P).$$

Multiplying $Z(P)$ by a factor that the spec attaches to $X(P)$ scales the two coordinates asymmetrically, and the translate produces a different cubical point. The error compounds across ladder iterations and CubicalRatio (Alg. 8.17), and surfaces as a non-bilinear "Tate pairing": the output is still a 2^e-th root of unity, but $T([2]P, Q) \neq T(P, Q)^2$. That alone breaks every downstream pairing-based check—in our case, the Weil-pairing codomain disambiguation in IdealToIsogeny that distinguishes the two factors of the $(2, 2)$-isogeny chain's output product (`dim2id2iso.c:1148-1178`). The chain quietly hands back "a curve and some basis points," downstream SplitAuxiliaryIsogeny fails the splitting condition with $\#\{i, j : U_{i,j}(0) = 0\} = 0$ instead of the unique 1, and the diagnosis points everywhere except at the CubicalDiffAdd optimization that introduced the asymmetry.

We discovered the bug only by writing a direct bilinearity test

$$T([2]P,\, Q,\, [2]P{+}Q) \stackrel{?}{=} T(P,\, Q,\, P{+}Q)^2,$$

which failed under both the $P{+}Q$ and $P{-}Q$ conventions for the third argument. The fix was a one-line revert of the "optimization" to the spec's literal form.

Advice for implementers: For x-only / cubical / theta arithmetic, transcribe the spec's formulas line by line on the first pass, including operand placement. Reject the urge to "save an inversion" by moving a factor between X and Z: the affine ratio is preserved, but the specific projective representative is load-bearing for the next algorithm. Add a bilinearity test for any pairing primitive—the unit-of-output check (the result lies in μ_n) passes for many non-bilinear "pairing-like" functions and is not enough. When profiling later identifies the inversion as a bottleneck, batch inversions across ladder steps via a Montgomery-style trick that preserves the projective rep at each step; do not rewrite the algebra.

4.18 Lattice ball sampling needs the dual-LLL approach

Algorithm 3.3 (LatticeSampling, used by Algorithm 4.3 RandomEquivalentQuaternion) is described abstractly: "sample a uniformly random element from the lattice L whose reduced norm is at most ρ." The spec does not specify which sampling algorithm to use; the C reference's `quat_lattice_sample_from_ball` (`lat_ball.c:58`) implements a specific algorithm that we initially missed:

1. Compute the primal Gram G of the lattice basis.
2. Compute the dual Gram $\mathrm{dual}G = \mathrm{adj}(G)$ and $\det G$.
3. LLL-reduce $\mathrm{dual}G$ *in place*, tracking the unimodular transformation U: $\mathrm{dual}G_{\mathrm{red}} = U^\top \cdot \mathrm{dual}G \cdot U$.
4. Set per-axis bounds $\mathrm{box}[i] = \sqrt{\mathrm{dual}G_{\mathrm{red}}[i][i] \cdot \rho / \det G}$.
5. Compute $U^{-1} = \mathrm{adj}(U) \cdot \mathrm{sign}(\det U)$ (integer since U is unimodular).
6. Sample $y[i] \in [-\mathrm{box}[i], \mathrm{box}[i]]$, compute $x = U^{-\top} \cdot y$, accept if $0 < x^\top G x \le \rho$.

Our initial port LLL-reduced the *primal* Gram and used the naive bound $\mathrm{box}[i] = \sqrt{\rho / G_{\mathrm{red}}[i][i]}$, which is correct but produces a much looser bounding parallelogram. The acceptance rate sat around 10^{-4}, requiring an inflated 200,000-iter inner budget just to converge.

The looseness was not subtle once we measured it: KAT vector 3 (a deterministic seed where the sign retry loop is sampling-bound) ran to the 360 s nextest budget and *never* produced a valid signature in our build, while C ref's signed in seconds. The trajectory after switching to the dual-LLL approach matched the C ref's per-iter behavior, and KAT 3 went from TIMEOUT to ~ 43 s on the same seed.

Advice for implementers: Algorithm 3.3 has more content than the spec text reveals. Specifically, the "LLL" step in the spec is on the *dual* Gram (LLL of $\mathrm{adj}(G)$ tracking U), and the box derived from $\mathrm{dual}G_{\mathrm{red}}[i][i]$ is what ties the bound to the ellipsoid's inertia tensor; the primal-Gram bound looks reasonable but is asymptotically off by the relevant Hermite-constant factor and makes a noticeable difference in the signing retry budget. Read `lat_ball.c` before implementing.

The fix is essentially trivial in code (a $\sim$50 LoC swap to `NrdBasis::from_cols_and_gram(identity, dualG).l2_reduce()` + a $U^{-\top}$ application after each sample), but expensive to identify without an external benchmark to compare against. We had the existing primal-LLL path running for weeks before the per-iter trajectory analysis pointed at sampling efficiency.

4.19 L^2 deep-insertion iteration order is implementation-defined

Algorithm 3.6 (L^2-DeepInsertion, used by every LLL reduction in the keygen pipeline through `quat_lideal_prime_norm_reduced_equivalent`) defines the deep-insertion point as "the smallest j such that $\bar{\delta} \cdot r[j][j] > t[j]$," i.e. the smallest level at which the L^2-condition fails on the partial squared-norm sequence $t[0], \ldots, t[\kappa - 1]$.

The C reference's `quat_lll_core` (`quaternion/ref/generic/lll/l2.c:110-114`) computes the insertion point s by a different iteration order:

```
for (swap = kappa; swap > 0; swap--) {
    if (delta_bar * r[swap-1][swap-1] < lovasz[swap-1])
        break;
}
```

That is, it iterates s *from* κ *down to* 1 and breaks on the first level where the L^2-condition still holds. Our initial port followed the spec literally:

```
let mut s = k;
for j in 0..k {
    if t[j] < delta_bar * r[j][j] {
        s = s.min(j);
```

```
    }
}
```

When the failure pattern is monotonic—fails for $j < a$, holds for $j \geq a$—both formulations return $s = a$. The partial squared norms $t[j]$ come from the running Gram–Schmidt orthogonalization (GSO) update $t[j] = t[j-1] - \mu[\kappa][j-1] \cdot r[\kappa][j-1]$, computed on floating-point scalars (`dpe_t` in the C ref, `DoublePlusExponent` in our port). Under rounding in those GSO scalars *the failure pattern can be non-monotonic* for some inputs: failures at, say, $j = 0$ and $j = 2$ but a hold at $j = 1$. The spec-literal form returns $s = 0$ (smallest failing level); the C ref returns $s = 2$ (largest level above the deepest contiguous hold).

Both are valid deep-insertion choices in the sense that each produces a properly L^2-reduced basis. They produce different reduced bases: the resulting bases differ in the sign of column zero (one formulation passes b_κ over a hold layer that the other does not, propagating opposite size-reduction signs). The post-LLL Gram matrices then differ in $G[0][2]$, $G[0][3]$ (and their transposes), with diagonals and other off-diagonals byte-identical.

Downstream consequences are not subtle. After the LLL-reduced basis enters `quat_lideal_prime_norm_reduce` (`lll_applications.c`), the rejection-sampling loop generates short-vector candidates c and accepts the first c whose class quadratic form $c^\top G_{\text{class}} c$ is prime. A different post-LLL G_{class} produces a different acceptance, hence a different equivalent prime ideal, hence a different chain codomain, hence a different public key. For 16 of our 100 NIST-I KAT vectors—every input that hit the non-monotonic condition—this single-line iteration-order divergence cascaded all the way to a public-key mismatch.

Fix. Match the C ref's iteration order exactly. The two-line change in `NrdBasis::l2_reduce` (`src/quaternions/lattice.rs`):

```
let mut s = k;
while s > 0 {
    if !(t[s - 1] < delta_bar * r[s - 1][s - 1]) {
        break;
    }
    s -= 1;
}
```

moved the keygen byte-equality count from 83/100 to 99/100 in a single commit, with no other changes.

Advice for implementers: Algorithm 3.6's English description is mathematically unambiguous when the L^2-condition behaves monotonically, but the algorithm is run on floating-point GSO scalars where monotonicity can fail by rounding error. When two equivalent descriptions of a step coincide "up to monotonicity", pick the one the reference chose, even if the spec's English suggests otherwise: KAT byte-equality requires matching the implementation, not the specification.

4.20 SplittingIsomorphism omits a side-channel randomization that the KAT vectors require

The output of an extra-torsion-free $(2, 2)$-isogeny chain is a level-2 theta null point on a product abelian surface $E_1 \times E_2$. Multiple projective representatives exist for the same abstract surface, related by the symplectic-modular-group action. The spec's SplittingIsomorphism (Algorithm 8.42) [**?**] picks one specific representative deterministically from the chain's terminal theta null—but that choice is a function of the secret kernel, so the emitted projective representative leaks information about the kernel.

The C reference closes this leak by pre-multiplying the splitting matrix M by a uniformly random $M_k \in \{\texttt{NORMALIZATION\_TRANSFORMS}[0..6)\}$, where the six matrices are

tensor products $M_k = m_k \otimes m_k$ of 2×2 coset representatives of the symplectic-modular-group quotient $\Gamma/\Gamma^0(4)$. The choice is sampled by reading four bytes of randomness from the deterministic byte-replayable RNG, parsing them as a little-endian `u32`, rejection-sampling against threshold 4,294,967,292 for an unbiased mod 6, and indexing the precomputed list (`theta_isogenies.c:980, 1043-1059`; matrices in `precomp/.../hd_splitting_transforms.c`).

Two consequences:

1. It is a real side-channel hardening. Without it, the public output of keygen and signing is a deterministic function of the secret kernel structure, not just the abstract codomain. An attacker observing the projective representative of a public key or signature could potentially recover information about the kernel's specific structure beyond what the curve isomorphism class already reveals.

2. The published KAT vectors are produced *with* this randomization applied. An implementation that skips the randomization will produce a public key that is projectively equivalent to the KAT vector's, but byte-different. Without mirroring the same RNG byte consumption pattern and matrix selection logic, byte-equality with the published KATs is impossible.

We mirror the C reference's randomization in `src/surfaces/precomputed.rs` (the six precomputed matrices) and `src/surfaces/isogeny.rs` (`sample_normalization _index` and the splitter integration). The randomization is gated on a caller-supplied `Option<&mut dyn rand_core::RngCore>`: `Some(rng)` for keygen's outer chain (matching C ref's `theta_chain_compute_and_eva` `None` for FDI's internal chains (which match C ref's deterministic `theta_chain_compute_and_eval`) and for verification.

Advice for implementers. Read the C reference's `splitting_compute` function (`theta_isogenies.c:1001`) end-to-end before implementing the splitter, including the `#if defined(ENABLE_SIGN)` block. Without it, your $(2,2)$-isogeny chain will produce projectively-equivalent but byte-different output, and the spec's KAT comparisons will fail in a way that misdirects effort toward looking for chain-internal bugs that aren't there. See also §**??** of the spec review.

5 Bugs from Fixed-Width Arithmetic

The C reference uses GMP, so its BigInts grow as needed. We use a fixed-width `BigInt<N>` type with N 64-bit limbs, sized per Kim et al. [**?**] for NIST-I worst-case bounds. Fixed widths give us constant-time execution and stack allocation, but they add a new failure mode: *silent overflow*. `ct_mul` at width N returns a `BigInt<N>` and drops any bits above limb N. If the caller doesn't check, later code sees a wrapped value and produces wrong answers with no warning.

5.1 `pow_mod` silently overflows for large moduli

`pow_mod` does modular exponentiation by square-and-multiply:

```
pub fn pow_mod(base: &Self, exp: &Self, modulus: &Self) -> Self {
    let mut result = Self::ONE;
    for bit in exp.bits().rev() {
        result = result.ct_mul(&result).ct_mod(modulus);
        if bit { result = result.ct_mul(base).ct_mod(modulus); }
    }
    result
}
```

After each reduction, `result` $<$ `modulus`. But before the reduction, the product `result` $\cdot$ `result` can be as large as (`modulus` $-1)^2$. For this to fit without truncation, we need

$$64N \geq 2 \cdot \text{bits}(\texttt{modulus}).$$

Our commitment modulus $D_{\text{mix}} = 2^{512} + 75$ is 513 bits. Squaring needs 1026 bits, or about 17 limbs. The natural storage is `BigInt<9>` (576 bits), which is one limb above the bit-length. Called at that width, `pow_mod` truncates every squaring to 576 bits and returns garbage.

The bug is deceptive because `is_probable_prime`, `legendre`, and `modular_sqrt` all call `pow_mod`. One wrong arithmetic result at the bottom makes all three lie:

- `is_probable_prime(D_mix)` returns `false`, even though D_{mix} is prime.
- `legendre(4, D_mix)` returns -1, even though 4 is a perfect square (so the answer must be 1).
- `modular_sqrt` never finds a root. `RandomIdealGivenNorm` rejects every sample. Key generation hangs.

It stayed hidden because earlier tests only called these functions at `BigInt<4>` with inputs under 128 bits—small enough that the squares fit. It surfaced the first time we ran `SigningKey::generate` and evaluated Euler's criterion at D_{mix} scale.

Immediate fix. In `random_prime_norm_wide`, widen the inputs to `BigInt<18>` (1152 bits) before the Legendre check and square root, then narrow the result back to `BigInt<9>`. 18 is the smallest width where $64N \geq 2 \cdot 513$, with room to spare.

Proper fix (tracked). `pow_mod` and its three callers should take a method-level const generic W for the working width, separate from the storage width N. Callers pass $W \geq 2N$ explicitly, enforced by a `const` assertion. The current single-width entry points still work for small moduli (under $32N$ bits), where N is already enough. Until we do this, every call site that passes a near-full modulus is potentially wrong and will fail silently.
Advice for implementers: With fixed-width arithmetic, every call site that chains multiplications needs a width budget. A function that takes a `BigInt<N>` is not self-contained: its correctness depends on N being big enough for its internal operations. Document the requirement in the rustdoc, and enforce it at compile time with const generics or a sealed trait when it's nontrivial. The worst bugs are the ones where a wrong value at the bottom of the stack flips a cryptographic predicate—"is this prime?", "is this a square?"— with no other symptom.

5.2 Six bugs in `GeneralizedRepresentInteger`

Algorithm 3.12 (GeneralizedRepresentInteger) finds $\gamma \in \mathcal{O}$ with $\text{nrd}(\gamma) = M$ by sampling (z, t) and solving $x^2 + qy^2 = M'$ via Cornacchia. Our implementation had six independent errors that together prevented the algorithm from ever succeeding during signing:

1. **Off-by-one in z sampling.** The spec says "sample z uniformly from $[1, \ldots, m]$" (line 4). Our loop variable started at 2 instead of 1, skipping the crucial $z = 1$ case. With z even, $M' = 4M - p(z^2 + qt^2)$ is always even and therefore never prime. The `is_probable_prime` check rejected every candidate, making the algorithm appear to find no solutions at all.

2. t **sampled from** $[0, m']$ **instead of** $[-m', m']$**.** The spec says "sample t uniformly from $[-m', \ldots, m']$" (line 5). We sampled only non-negative values. Since M' depends on t^2, the Cornacchia output (x, y) is the same for $\pm t$, but the isogeny condition (line 15, $x - t \equiv 2 \pmod 4$) depends on the sign of t. Missing negative t values halves the chance of satisfying this congruence.

3. **Incorrect divisibility-by-2 check.** The spec constructs $\gamma = x + \omega y + jz + \omega jt$ and checks that the largest d with $\gamma/d \in \mathcal{O}$ equals 2 (lines 18–19). We initially replaced this with an "all coordinates same parity" check in $\{1, i, j, k\}$, then with a hand-derived congruence condition. Both were wrong.

   The correct approach, matching the C reference's `quat_alg_make_primitive`, is to construct γ as a quaternion element in the order, compute the GCD of its coordinates in the order's integral basis, and verify the GCD equals 2. Attempting to reason about divisibility-by-2 in the $\{1, i, j, k\}$ coordinates is fragile because the order $\mathcal{O}_0$ has a basis with half-integer entries ($\frac{1+j}{2}$, $\frac{i+k}{2}$); divisibility by 2 in the order is not equivalent to any simple parity condition on the $\{1, i, j, k\}$ coordinates.

4. **Arithmetic overflow in primality and Cornacchia.** The candidates M' are ∼273-bit values stored in `BigInt<8>` (512 bits). Both `is_probable_prime` and `cornacchia` internally call `pow_mod`, whose squaring step produces ∼546-bit intermediates that overflow the 512-bit storage. The result is that `is_probable_prime` rejects genuine primes and `cornacchia` returns `None` for valid inputs. Fixed by using the widened variants `is_probable_prime_w::<9>` and `cornacchia_w::<9>` (576 bits $\geq 2 \times 273$).

5. **Hardcoded search bound too small.** We initially used `bound = 256`. The spec computes the bound as $\lceil \sqrt{4M/(p\sqrt{q})} \rceil$, which for NIST-I at the FixedDegreeIsogeny call site is ∼1400–2500. With 256 iterations, the search exhausted without finding any (z, t, x, y) satisfying both the isogeny condition and the $d = 2$ condition. Fixed by computing the bound from the spec's formula.

6. **Primality width too narrow for the auxiliary call site.** Bug 4 was patched by moving `is_probable_prime` and `cornacchia` to the widened variants `is_probable_prime_w::<9>` and `cornacchia_w::<9>`. That width ($9 \times 64 = 576$ bits) covers the FixedDegreeIsogeny caller's ∼273-bit M' with margin, but it does not cover the *response-phase* auxiliary caller `random_norm` (§5.1), which passes $M = \text{QUAT\_PRIME\_COFACTOR} \cdot N_{\text{aux}} \approx 2^{377}$. At that magnitude $4M$ is ∼514 bits, needing $64W \geq 2 \cdot 514 = 1028$, so $W \geq 17$. At the original $W = 9$, the Miller-Rabin squarings overflowed 576 bits on every candidate M', `is_probable_prime_w` reported "composite" for every prime, and `represent_integer` looped indefinitely without ever finding a witness. From signing's point of view the aux path never completed: `sign` ran through its 1000-iteration retry bound and returned `SigningFailed`.

   The fix is to widen both callers to `is_probable_prime_w::<17>` and `cornacchia_w::<17>`, which covers every `BigInt<8>` input ($1088 \geq 2 \cdot 514$). With the wider working width, `represent_integer` at aux-path magnitudes completes in ∼7 seconds.

   Structurally this is a direct recurrence of bug 4 and of the underlying width-budget issue in §5.1: our machinery for expressing the "working width must cover $2 \cdot$ bits(input)" contract is a compile-time `const` assertion on $W \geq N$ only, not on W versus the actual input bit-length. The const assertion lets $W = 9$ through even when the actual inputs are 514 bits. We added a regression test (`primality_w9_vs_w20_at_aux_magnitude`) that confirms narrow-W Miller-Rabin disagrees with wide-W on 379-bit candidates, so future width regressions surface immediately.

Bugs 1, 4, and 6 were the most damaging: bug 1 eliminated all candidates before primality testing, and bugs 4 and 6 made the primality test reject genuine primes. Bug 3 was the subtlest and required the most iterations to diagnose, since the divisibility condition depends on the order's algebraic structure rather than a simple coordinate check. Bug 6 was particularly hard to catch because it only manifested at the aux-path call site, several layers below `sign`, and the outer failure mode (`sign` exhausts its retry budget and returns `SigningFailed`) looks indistinguishable from the normal probabilistic failure of SuitableIdeals.

Advice for implementers: Algorithm 3.12 is simple at first glance—a short loop with Cornacchia and a divisibility check—but the implementation details (sampling ranges, sign conventions, order-basis divisibility, arithmetic width, and search bounds) are all places where a plausible-looking implementation silently fails. The divisibility check in particular should not be hand-derived from coordinate arithmetic; it should use the same order-basis GCD computation as the C reference.

5.3 `random_norm` narrows $\mathrm{nrd}(\beta)$ into `BigInt<4>`

Algorithm 3.10 (RandomIdealGivenNorm) constructs an ideal of a given norm N by sampling a random quaternion $\beta = x + yi + zj + wk$ with coordinates in $[1, N]$ and checking $\gcd(\mathrm{nrd}(\beta), N) = 1$. Our implementation computed $\mathrm{nrd}(\beta)$ at `BigInt<8>` (the width of the norm formula $x^2 + y^2 + p(z^2 + w^2)$), then tried to narrow it to `BigInt<4>` before computing the gcd:

```
let (nrd_num, nrd_den) = beta.norm();
let nrd_num_4: CtOption<BigInt<4>> = nrd_num.into();
let nrd_den_4: CtOption<BigInt<4>> = nrd_den.into();
if !nrd_num_4.is_some() || !nrd_den_4.is_some() {
    continue;  // retry with fresh sample
}
```

For $N \approx 2^{126}$ (signing response phase) the norm $\mathrm{nrd}(\beta) \approx p \cdot N^2 \approx 2^{505}$. That doesn't fit in `BigInt<4>`'s 256 bits, so the narrow always failed and every one of the 10 000 samples was silently rejected. `random_norm` always returned `None`; signing's auxiliary path always continued past it; key generation and signing appeared to "probabilistically fail" when in fact they were deterministically wrong.

Fix. Widen N to `BigInt<8>` and compute the gcd at that width. The result is always small (bounded by N), so the final comparison against 1 is correct regardless of storage width.

Why it was hidden. `random_norm` is only called from the signing response path's auxiliary ideal construction. Key generation and signing's commitment phase use `random_prime_norm_wide`, which operates at `BigInt<30>` throughout and doesn't narrow. Unit tests for `random_norm` used small N (under 2^{60}), where the narrow accidentally succeeded. The bug only surfaced when the signing flow first reached the aux path after we had fixed the upstream intersection-overflow and outer-chain-torsion bugs — at which point `random_norm` quietly rejected every sample and sign looked like it was "stuck in the response phase."

Advice for implementers: A `CtOption`-returning narrow at the edge of a module is a silent-failure channel. Any time you narrow a computed-at-large-width quantity into a smaller storage type on the hot path, write a unit test that passes an input large enough to make the narrow fail and verify the outer function still behaves correctly. When the narrow is meant to prove "this value fits," make it a `Result` with a distinguishable error rather than a filter that loops until it happens to fit.

Related: narrowing wide ideals to `LeftIdeal<4>`. The signing response phase constructs the ideal $I_{\text{com,rsp}}$ with norm $\sim 2^{258}$ at storage width `BigInt<30>`. Passing it to IdealToIsogeny requires a `LeftIdeal<4>`, but the HNF basis diagonal just barely exceeds 2^{256} so direct narrowing fails. An earlier version fell back to `reduce_to_prime_norm::<44>`, which took ∼40 s per call and usually failed. We replaced the fallback with a generalized smallest-equiv reduction (`LeftIdeal<N>::smallest_equiv_narrow<W>`) that LLLs the lattice at working width W, takes the shortest basis vector δ at full width, and returns $I \cdot \bar{\delta}/\text{nrd}(I)$ narrowed to `LeftIdeal<4>`. The resulting equivalent has norm $\sim\sqrt{p} \approx 2^{126}$, fits in `BigInt<4>` cleanly, and takes ∼1 ms per call. Keeping δ at the working width W rather than narrowing it up front was essential: short vectors of the input lattice can have coordinates up to the input-entry size ($\sim 2^{258}$), so narrow-to-4 on δ would have rejected every valid input.

5.4 `random_norm` off-by-one in β rejection sampling

The same Algorithm 3.10 (RandomIdealGivenNorm) calls for $\beta = x + yi + zj + wk$ with coordinates uniformly in $[1, N]$, the same range the C reference draws via `ibz_rand_interval(out, 1, norm)`. We had reimplemented the rejection-sampling loop inline rather than reusing our existing `BigInt::rand_interval`. The inline version read $\lceil \log_2 N/8 \rceil$ bytes, applied the same top-byte mask the C reference uses, but then rejected `val` $= 0$ and returned `val` directly:

```
loop {
    rng.fill_bytes(&mut bytes[..n_bytes]);
    if n_bits % 8 != 0 {
        bytes[n_bytes - 1] &= (1u8 << (n_bits % 8)) - 1;
    }
    let val = BigInt::<4>::from_bytes_le_unsigned(&bytes[..n_bytes])
        ;
    if bool::from(val.is_zero()) || val.ct_mod(n) != val {
        continue;
    }
    return val;
}
```

The C reference instead accepts `tmp` $\leq N - 1$ and returns `tmp` $+ 1$:

```
do {
    randombytes((unsigned char *)r, len_bytes);
    r[len_limbs - 1] &= mask;
    mpz_roinit_n(tmp, r, len_limbs);
    if (mpz_cmp(tmp, bmina) <= 0)  // bmina = N - 1
        break;
} while (1);
mpz_add(*rand, tmp, *a);  // *a = 1
```

Per attempt the byte consumption is identical: both sides draw the same $\lceil \log_2 N/8 \rceil$ bytes, both apply the same top-byte mask, and the reject conditions (`val` $\geq N$ on our side, `tmp` $> N - 1$ on the reference's) coincide on every nonzero value. But the C reference's *accepted* value is exactly one larger than ours: on a shared DRBG, each of the four coordinates of β came out one less than the reference's, except in the vanishing-probability case where the bytes are all zero (which the reference accepts and remaps to 1).

This mistake interacted badly with what came after. After sampling β, the algorithm computes $\alpha = \gamma\beta$ where γ has reduced norm $m \cdot N$ from RepresentInteger. A constant shift of the coefficients of β by $(1, 1, 1, 1)$ propagates through the non-linear quaternion

multiplication as

$$\alpha_{\text{ours}} \;=\; \gamma\beta_{\text{ours}} \;=\; \gamma\big(\beta_{\text{ref}} - (1+i+j+k)\big) \;=\; \alpha_{\text{ref}} \;-\; \gamma\cdot(1+i+j+k),$$

and $\gamma\cdot(1+i+j+k)$ generally does *not* lie in the right-ideal $O_0\cdot N$, so the resulting O_0-ideal $O_0\cdot\alpha + O_0\cdot N$ is a different $\mathbb{Z}$-module than the reference's, not just a different basis representation of the same module. The intersection $I_{\text{inter}} = I_{\text{com,rsp}} \cap I_{\text{aux}}$ inherits the divergence, and the response-phase IdealToIsogeny on I_{inter} ran an L^2-LLL on a basis that no longer matched the reference's at byte level.

Diagnosis. We built a cross-implementation L^2-LLL dump harness: a `SELKIE_L2_DUMP_DIR` env variable in our `l2_reduce` writes each L^2 input (G, B) to a numbered text file, and we added the same `CREF_L2_DUMP_DIR` hook to the C reference's `quat_lll_core`. On the same KAT seed, hashing the files and joining by content matched the first seventeen L^2 calls byte-for-byte and diverged on the eighteenth. Caller-location tags written to a sister `l2_caller_NNNN.txt` (using `#[track_caller]` on `l2_reduce` and the `core::panic::Location::caller()` API) identified call 18 as the $t = 0$ extremal-order step of IdealToIsogeny's SuitableIdeals, run on I_{inter}. Dumping the HNF bases of I_{aux} from both sides then showed off-diagonal column entries differing by amounts unrelated to the lattice norm — a different $\mathbb{Z}$-module, not just a different basis.

Fix. Mirror the C reference's mapping exactly. Sample `tmp` from $[0, N-1]$ via top-byte-masked rejection on $\lceil \log_2(N-1)/8 \rceil$ bytes, reject `tmp` $> N-1$, and return `tmp` $+ 1$. This made `random_norm`'s I_{aux} byte-equal to the reference's, the downstream intersection byte-equal, and the two previously-stuck KATs (#39 and #80, which had been exhausting in-iter rejection samplers for tens of seconds without ever finding a valid response α) returned in ~ 2.5 s each, succeeding on iteration 0 like the reference. After the fix, 94 of 100 signing KATs pass; the remaining six (#20, #53, #56, #79, #94, #95) fail with a distinct lattice-basis divergence in `from_generator_mod_hnf`'s canonical-HNF reduction, unrelated to this sampling bug.

Advice for implementers: When emulating a C-reference sampling primitive on a shared DRBG, the byte-stream contract is half the story. The other half is the *mapping* from raw bytes to the returned value: an inclusive vs. exclusive bound, a $+a$ offset, or a different rejection threshold all produce the same byte consumption and the same distribution but different individual samples. For KAT byte-equality, the mapping has to match exactly — ideally by routing all such sampling through a single shared primitive (we already had one, `BigInt::rand_interval`; this loop just hadn't been refactored onto it). When that's infeasible, write a tiny property test that pulls bytes from a fixed-seed DRBG and verifies your routine and the reference's `ibz_rand_interval` return identical sequences for the same range $[a, b]$.

5.5 Modular HNF modulus must be a lattice member, not just a covolume bound

Algorithm 3.16's id2iso decomposition (SuitableIdeals) enumerates short vectors in the seven extremal-order parents $I^{(t)} = \text{conj}(I_{\text{reduced}})\cdot J_t$, where J_t is a precomputed connecting ideal. Computing $I^{(t)}$ as a lattice product on the integer columns of I_{reduced} and J_t involves an HNF reduction of the 16 generators $(b_i^{I_{\text{reduced}}} \cdot b_j^{J_t})_{i,j}$.

Our `Lattice::product_with_modulus` (used to specialize the HNF reduction with a precomputed covolume bound) calls `Matrix::from_hnf_columns_mod`, whose contract on the modulus D is stricter than "D is a multiple of $\det(L)$." The implementation appends explicit columns $D\cdot e_i$ to the generator list (Cohen's Algorithm 2.4.8) and HNF-reduces

the union $L + D \cdot \mathbb{Z}^4$. That equals L only when $D \cdot e_i \in L$ for every i — equivalently, when D is a lattice *member*, not merely a covolume bound.

We supplied $D = 1024 \cdot N^4 \cdot (k \cdot N_J)^2$, intended as the canonical integer-column covolume of the product lattice at denominator $4N$. For the default response case (denominator $4N$ on I_{reduced} and denominator 2 on J_t) the actual canonical covolume is $(8N)^4 \cdot (kN_J)^2 = 4096 \cdot N^4 \cdot (kN_J)^2 = 4 \cdot D$, so our D is a strict divisor of the covolume — and is not a lattice member for these inputs. The HNF returns the strictly coarser super-lattice $L + D \cdot \mathbb{Z}^4$, whose canonical generator includes the integer scalar $(2, 0, 0, 0)$.

That scalar then enters the class Gram $g_{ij} = 2\,\text{nrd}(b_i, b_j)/(d^2 \cdot N(I))$ used by L^2-LLL. The first column has $\text{nrd}((2, 0, 0, 0)) = 4$, while $d^2 \cdot N(I) \approx 2^{770}$ at width 16, so integer division floors g_{00} (and all of row 0) to zero. Both our `l2_reduce` and the C reference's `quat_lll_core` non-terminate on that Gram: the DPE r values collapse to ± 0 (mantissa ± 0, exponent -2^{31}) and the Lovász/swap test enters an infinite swap loop. We verified this is not specific to our implementation by feeding the dumped degenerate Gram into a C-reference harness that calls `quat_lll_core` directly: it also runs at 100% CPU for > 5 minutes without producing output. The reference doesn't hit this state during `PQCgenKAT_sign` because its cross-order construction (`quat_lideal_lideal_mul_reduced` → `quat_lattice_mul` in `lattice.c:202-244`) uses $|\det(\text{first 4 product generators})|$ as the HNF modulus (`lattice.c:231-233`):

```
ibz_mat_4x4_inv_with_det_as_denom(NULL, &det, &detmat);
ibz_abs(&det, &det);
ibz_mat_4xn_hnf_mod_core(&(res->basis), 16, generators, &det);
```

`detmat` here is the matrix of the first four generators; its $|\det|$ is the covolume of those four generators, which lies in the lattice they span. Our `Lattice::product` (without the explicit modulus) mirrors that exactly.

Fix. Switch the cross-order step at `src/quaternions/ideal.rs:1873` from `conj_lat.product_with_modulus(&
&modulus_outer)` to `conj_lat.product(&j_t_lat)`, which uses `Lattice::product`'s built-in $|\det(\text{first 4 cols})|$ modulus. This is the C-reference recipe. After the fix, all six previously-stuck KATs (#20, #53, #56, #79, #94, #95) complete in 1–6 seconds on iteration 0, and 100 of 100 signing KATs pass.

Why it was hidden. For most response-phase ideals the precomputed bound D happens to coincide with the canonical covolume, so $D \cdot e_i \in L$ and the result is correct. KAT 53 in particular produces an I_{inter} whose post-LLL basis has a non-canonical HNF (`basis[2][2] = 5`, `basis[2][3] = 2`) — the lattice content includes a factor of 5 in row 2. This shifts the denominator structure so that our hard-coded $1024 \cdot N^4$ factor no longer matches the actual covolume, D becomes a strict divisor of the canonical value, and the HNF widens to the coarser super-lattice. KATs 39 and 80 don't hit this because their off-by-one in β sampling (§5.4) had already broken I_{inter} upstream, so they never reached this code path with the troublesome geometry.

Advice for implementers: When using a modular HNF algorithm (Cohen's Algorithm 2.4.8) to compute lattice products, the modulus must be a lattice member, not just an arbitrary multiple of the determinant. The two conditions are equivalent for self-dual lattices but distinct in general — a D that is $k \cdot \det(L)$ for $k > 1$ may or may not live in L. When in doubt, do what the C reference does: compute $|\det(\text{first 4 generators})|$ as the modulus rather than trying to derive a covolume formula. Document the membership condition prominently on any function whose contract claims "modulus must be a multiple of covolume" — that's not sufficient, and the failure mode (coarser lattice → degenerate Gram → non-terminating L^2) takes hours to debug.

5.6 Invalid test kernels for $(2,2)$-isogeny chains

This was the most time-consuming false lead we encountered. We wrote three unit tests that exercised the $(2,2)$-isogeny chain on short chains ($e = 2, 3$) with synthetic kernel points constructed from arbitrary basis elements — for example, $(P, Q) \times (Q, P)$ on $E_0 \times E_0$, where P, Q are torsion basis points. All three consistently failed at the splitting step (Algorithm 8.41), which found zero vanishing $U_{i,j}(0)$ coordinates instead of the expected one.

Because 100 KAT verification vectors exercised the same chain code and passed, we assumed the test data was valid and spent considerable effort searching for a bug in our implementation: comparing every intermediate value against the C reference, tracing Hadamard transform conventions through the chain, checking projective representatives from Jacobian doubling, and adding isotropicity diagnostics. None of these investigations found a discrepancy — the formulas matched the C reference exactly.

We had originally pulled these kernel constructions from the C reference implementation's test suite without modification or independent validation. Because the C reference exercises the $(2,2)$-chain only through the full signing/verification flow (which produces valid kernels by construction), its tests never exercise synthetic kernels of this form. We adopted the kernel patterns without checking whether they satisfied the theta formula preconditions.

The test data was invalid. The synthetic kernels are *isotropic* in the Weil-pairing sense (the product Weil pairing is trivial), but they cause the `ActionByTranslation` determinant $\delta = x_{P_4} z_{P_2} - z_{P_4} x_{P_2}$ to vanish. The theta change-of-basis (Algorithm 8.36) requires inverting δ; when $\delta = 0$ the formulas silently produce wrong output. We confirmed this by running the same kernels through the DMPR24 reference SageMath implementation [**?**], which also fails with a `ZeroDivisionError` in the same batched inversion.

The specification does not document the nonvanishing-δ precondition. The C reference avoids it in practice because its kernels come from FixedDegreeIsogeny (Algorithm 3.15), which constructs kernels from endomorphisms produced by RepresentInteger — these structured kernels have nonzero δ by construction. When δ does vanish (a rare edge case), the C reference's `gluing_compute` returns failure and the signing loop retries.

Advice for implementers: Not every isotropic subgroup of $E_1[2^e] \times E_2[2^e]$ is a valid kernel for the theta $(2,2)$-isogeny formulas. The formulas additionally require nonvanishing of certain determinants in the theta change-of-basis computation. This precondition is nowhere stated in the specification and is not obvious from the pseudocode. We wasted significant debugging time because our test data violated an undocumented assumption. The fix was to delete the invalid tests and rely on KAT verification (100 vectors, all passing), which exercises the chain on kernels produced by the actual protocol flow.

5.7 HNF coefficient blow-up at fixed precision

The commitment phase constructs the left ideal $I = O_0\langle \gamma, N \rangle$ by concatenating the eight column vectors of $O_0 \cdot \gamma$ and $O_0 \cdot N$ and reducing to Hermite Normal Form (Algorithm 3.2). The C reference uses GMP, so HNF intermediate entries grow without bound. We use fixed-width `BigInt<30>` (1920 bits).

Classical HNF's extended-GCD elimination produces intermediate column entries whose magnitude can grow exponentially in the rank. For our rank-4 ideal lattices with initial entries up to $\sim 2^{770}$ (the $p \cdot g_i$ products from `RandomIdealGivenNorm`), intermediate xgcd products exceed the 1920-bit budget after only a few elimination steps. The overflowing limbs are silently truncated, and the resulting "HNF" basis collapses: we observed basis columns with reduced norms of 4 and $\sim 2^{251}$—elements of the ambient order O_0, not the intended ideal. Every downstream operation then computes on a lattice unrelated to I, and `reduce_to_prime_norm` never finds a valid prime because the divisibility invariant

$\mathrm{nrd}(\alpha) \in N(I) \cdot \mathbb{Z}$ no longer holds.

The fix is the standard Domich–Kannan–Trotter modular HNF (Cohen, *A Course in Computational Algebraic Number Theory*, §2.4.2). If D is any positive multiple of the lattice determinant $\det(L)$, then $L \supseteq D \cdot \mathbb{Z}^n$, so reducing every intermediate entry modulo D after every arithmetic update preserves the HNF while bounding entries by D.

For the commitment ideal at NIST-I, $D = 4d^4N^2p \approx 2^{1282}$ (21 limbs) is a precomputed constant. The modular HNF operates at a wider intermediate width ($W = 44$ limbs, 2816 bits) to hold products of two D-sized values before reduction, then narrows the result back to the storage width. The same technique is applied to two additional HNF sites:

- **Response-phase ideal construction** (`from_generator` at width 30): the generator α_{rsp} from the sampling step has coordinates up to $\sim 2^{1400}$ bits; the modulus is computed per call as $4d^4 \cdot (norm)^2 \cdot p$.
- **Post-reduce basis update** inside `reduce_to_prime_norm`: the new basis columns are quaternion products with entries up to $\sim 2^{900}$ bits and integer-column covolume up to $\sim 2^{3000}$ bits. The modulus and intermediate arithmetic are widened to `BigInt<50>` and `BigInt<100>` respectively.

Advice for implementers: Algorithm 3.2 of the SQIsign specification describes HNF over arbitrary-precision integers; it does not discuss fixed-precision adaptations or coefficient-size bounds. Any fixed-width implementation of Algorithm 3.2 must either prove that intermediate growth stays within the budget for the specific input magnitudes, or use modular HNF with a known multiple of the lattice determinant as the bounding modulus. The C reference sidesteps this entirely by using GMP.

5.8 Missing u^{-1} normalization in FixedDegreeIsogeny

Algorithm 3.15 (FixedDegreeIsogeny) computes $\theta \leftarrow \mathrm{RepresentInteger}(u \cdot (2^e - u), \mathcal{O}_t, \mathrm{true})$ on line 2, then constructs a $(2,2)$-isogeny kernel from $([u]P, \theta(P))$ and $([u]Q, \theta(Q))$ on line 4. Since $\mathrm{nrd}(\theta) = u \cdot (2^e - u)$, the endomorphism θ has degree $u \cdot (2^e - u)$, and the actual kernel of the degree-$(2^e - u)$ isogeny we want is generated by $(P, (\theta/u)(P))$—not by $([u]P, \theta(P))$.

The spec's pseudocode says to form the kernel from $([u]P, \theta(P))$ and double $f - 2 - e$ times. This is algebraically equivalent to using $(P, (\theta/u)(P))$ only if the u-torsion is killed by the doublings, which it is. However, in practice, applying θ directly (without dividing by u) produces kernel points whose ActionByTranslation determinant is degenerate. Every one of the hundreds of $(2,2)$-chains we attempted failed with either zero or ten vanishing $U_{i,j}(0)$ indices (the correct count for a valid product kernel is exactly one).

The C reference (`dim2id2iso.c`, lines 133–143) multiplies all four quaternion coordinates of θ by $u^{-1} \bmod 2^{e+2}$ before applying the endomorphism to the torsion basis:

```
ibz_mul(&two_pow, &two_pow, &ibz_const_two);
ibz_mul(&two_pow, &two_pow, &ibz_const_two);
ibz_invmod(&tmp, u, &two_pow);
ibz_mul(&theta.coord[0], &theta.coord[0], &tmp);
ibz_mul(&theta.coord[1], &theta.coord[1], &tmp);
ibz_mul(&theta.coord[2], &theta.coord[2], &tmp);
ibz_mul(&theta.coord[3], &theta.coord[3], &tmp);
```

This effectively applies θ/u (mod the torsion order) to the basis, so the kernel is $(P, (\theta/u)(P))$ as intended. The modulus 2^{e+2} suffices because the kernel lives in $E[2^e]$ and the extra $+2$ accounts for the gluing's order-4 structure. Without this step, the inner $(2,2)$-chain *always* fails.

Advice for implementers: Algorithm 3.15 should either (a) state that the kernel is formed from $(P, (\theta/u)(P))$ and explain the modular inversion, or (b) prove that the

$([u]P,\, \theta(P))$ formulation produces a non-degenerate kernel without normalization. The current pseudocode silently requires a step that is absent from the algorithm box.

5.9 Sign-magnitude to unsigned modular conversion

The quaternion algebra's `decompose` function returns coefficients as signed integers (e.g., $c_0 = -8$ for $\theta = 3 + 5i + 7j + 11k$ in $\mathcal{O}_0$). These coefficients are used as scalars in the biladder to compute $\theta(P) = c_0 \cdot P + c_1 \cdot M_{\text{gen2}} + \cdots$ on the torsion basis $E_t[2^f]$.

Our `BigInt` type uses sign-magnitude representation, while `Scalar` (the biladder's input) is unsigned modular ($\bmod 2^{256}$). The `From<BigInt> for Scalar` conversion originally took the raw limbs without checking the sign bit, mapping -8 to $+8$ instead of $2^{256} - 8$. This produced the wrong endomorphism image for *every* nontrivial quaternion element from RepresentInteger, while passing tests that used simple elements like i or scalar multiples (whose decomposition has no negative coefficients).

Advice for implementers: Every boundary between signed and unsigned integer representations is a potential sign-swallowing bug. The type system did not prevent this because both `BigInt` and `Scalar` store the same `[u64; 4]` limbs — only the interpretation differs.

5.10 IdealToIsogeny outer scaling uses the reduced ideal's norm

Algorithm 3.13 (IdealToIsogeny) line 6 scales the matrix that builds the second component of the outer kernel by $1/(\text{nrd}(I)\,\text{nrd}(J_t))$, where I is the input ideal. The β_i and d_i returned by SuitableIdeals (Algorithm 3.16) satisfy $\text{nrd}(\beta_i) = d_i \cdot \text{nrd}(J_i\, I)$. An implementation that reads these literally and computes the scaling as $1/\text{nrd}(I)$ is correct *only if* β_i and d_i were computed with respect to the caller-supplied I.

Efficient implementations (and the C reference, `dim2id2iso.c:534-565`) first replace I with its smallest equivalent $I' = I \cdot \widehat{\delta}/\text{nrd}(I)$ to make the short-vector search tractable. The enumeration then returns β_i and d_i relative to I', so $\text{nrd}(\beta_i) = d_i \cdot \text{nrd}(I')$, which is not equal to $d_i \cdot \text{nrd}(I)$ in general. We hit this: every outer-chain attempt printed

$$\det(M_{\beta_1}) \pmod{2^f} \;\neq\; d_1 \cdot \text{nrd}(I) \pmod{2^f},$$

with the mismatch tracking the ratio $\text{nrd}(I')/\text{nrd}(I)$ across KAT inputs.

Two fixes produce the same behaviour:

- **Send β back.** After SuitableIdeals picks the pair, rewrite $\beta_i \leftarrow \delta\beta_i$ so that the returned β_i is in the caller's I. The C reference does this at `dim2id2iso.c:670-684` so that downstream callers can use $1/\text{nrd}(I)$ literally.
- **Track the provenance.** Store the reduced ideal alongside each returned β_i (as an `IdealFactor` field carrying the lattice β lives in), and have the caller read the scaling norm off that field. This is what `sqisign-selkie` does.

We chose provenance-tracking because it keeps SuitableIdeals free of the δ-multiplication step and localizes the invariant $\text{nrd}(\beta) = d \cdot \text{nrd}(\texttt{parent\_ideal})$ to the `IdealFactor` type. Adding the $\det(M_{\beta_1}) \stackrel{?}{=} d_1 \cdot \text{nrd}(\texttt{parent\_ideal}) \pmod{2^f}$ check as a diagnostic in test builds flagged the mismatch immediately.

Advice for implementers: Algorithm 3.13 line 6's $\text{nrd}(I)$ must be the norm of the *same* ideal whose lattice produced β_i. If SuitableIdeals substitutes a reduced equivalent of I, the downstream scaling must use the reduced ideal's norm (or β must be transported back to I before leaving the algorithm). Compare $\det(M_{\beta_1}) \pmod{2^f}$ to $d_1 \cdot \text{nrd}(\cdot) \pmod{2^f}$ on every call in test builds: a mismatch catches this immediately and the check costs one matrix determinant.

5.11 Connecting-ideal norm: leading basis entry vs. reduced norm

The seven extremal maximal orders $\mathcal{O}_t$ ($t = 0, \ldots, 6$ for NIST-I) come with precomputed connecting ideals $J_t \subseteq \mathcal{O}_0$ whose right order is conjugate to $\mathcal{O}_t$. Each J_t is stored as

- a basis matrix $B_t \in M_4(\mathbb{Z})$,
- a denominator $d_t \in \mathbb{Z}_{>0}$ (with $d_t = 2$ for the NIST-I orders),
- a labelled norm $N_t \in \mathbb{Z}_{>0}$,

where the rational basis vectors are the columns of B_t/d_t. The labelled norm N_t is supposed to be the *reduced norm* $N(J_t) = \sqrt{[\mathcal{O}_0 : J_t]}$; the C reference's `quat_lideal_norm` (`quaternion/ref/generic/ideal.c:7`) computes it from the lattice index. For our HNF shape $B_t = \mathrm{diag}(2N_t, 2N_t, 1, 1)$ at $d_t = 2$, the reduced norm is N_t and the leading basis entry is $2N_t$; the two differ by exactly the denominator.

Symptom. Our extraction script `scripts/precomp/extract_connecting_ideals.sage` computed

```
norm_row = int_mat.row_space(ZZ).intersection(
    (ZZ**4).submodule([[1, 0, 0, 0]])
).basis()[0]
norm = ZZ(abs(norm_row[0]))
```

which returns the leading first-coordinate entry of the integer HNF row space—i.e., $2N_t$, not N_t. Every entry of `CONNECTING_IDEAL_NORMS[t]` for $t = 0, \ldots, 6$ was therefore exactly twice the C reference's labelled norm.

This was latent for 99 of the 100 NIST-I lvl1 KATs because SuitableIdeals usually finds a viable (β_s, β_t) pair with $s = t = 0$ (the special-order path), which never reads `CONNECTING_IDEAL_NORMS`. KAT 29 was the first KAT whose hypercube enumeration at $m = 2$ exhausted the special order and forced the cross-order branch with $t = 1$, where the doubled norm flowed into two downstream divergences:

1. **`enumerate_short_vectors` divisor doubled.** The ideal-membership integrality check uses divisor $= N_t \cdot d_t^2$. With N_t doubled, half of the hypercube candidates fail integrality (those with odd isogeny degree are rejected; those with even degree are accepted with the degree silently halved). The sorted batch then differs from the C ref's, and `try_find_uv` picks a different (d_s, d_t) pair.
2. **IdealToIsogeny scale_denom doubled.** The outer-chain scaling $1/(\mathrm{nrd}(I) \cdot d_1 \cdot N(J_t))$ is multiplied by N_t for $t \neq 0$ (`deuring/mod.rs:895`). With N_t doubled the scale is also doubled, breaking the θ-on-the-second-component matrix entries mod 2^f.

Fix. Replace the leading-entry extraction with a direct lattice-index computation that matches `quat_lideal_norm`:

$$N(J_t)^2 \;=\; \frac{|\det(B_t)|}{|\det(B_{\mathcal{O}_0})|} \cdot \left(\frac{d_{\mathcal{O}_0}}{d_t}\right)^4$$

where $B_{\mathcal{O}_0}$ and $d_{\mathcal{O}_0} = 2$ are the standard order's basis and denominator. This is also what Sage's `QuaternionFractionalIdeal.norm()` returns; we use the explicit formula because the script holds only the basis matrix at this point. After regeneration, all seven entries match the C reference's `CONNECTING_IDEALS[t].norm` byte-for-byte. The `connecting_ideal()` runtime constructor was simultaneously updated to use $2 \cdot N_t$ as the leading basis entry, keeping the constructed lattice byte-unchanged while the labelled norm is corrected.

Catching this earlier. A self-consistency assertion in `connecting_ideal()` comparing the labelled norm to $\sqrt{[\mathcal{O}_0 : J_t]}$ (computable from the basis at construction time) would have flagged this immediately. None of our 99 working keygen KATs exercised the cross-order code path, so unit tests on individual algorithms didn't catch it either.
Advice for implementers: For Hermite Normal Form representations of fractional ideals, the leading basis entry is the *ideal generator* (e.g., N in $N \cdot \mathcal{O}_0 + \cdots$), not the reduced norm in general. The two coincide only at denominator 1. When porting from a representation with $d > 1$, compute the reduced norm via the lattice index, not by reading off a basis pivot.

5.12 Width-4 silent overflow in `Lattice::decompose` for $q \geq 5$ orders

Algorithm 3.12 (RepresentInteger) and the action-matrix construction in Algorithm 3.13 both decompose a quaternion element $\gamma \in B$ on the basis of an extremal maximal order $\mathcal{O}_t$:

$$\gamma \;=\; c_0 b_0 \;+\; c_1 b_1 \;+\; c_2 b_2 \;+\; c_3 b_3 \qquad (c_i \in \mathbb{Z}).$$

Our `Lattice::decompose` (`quaternions/lattice.rs:520`) solves for the integer coefficients c_i via Cramer's rule:

$$c_i \;=\; \frac{(\mathrm{adj}(B) \cdot v)_i}{\det(B)}$$

where B is the lattice basis matrix (with rows = coordinate components, columns = basis vectors), v is the target's coordinate vector scaled to a common denominator, and $\mathrm{adj}(B)$ is the 4×4 adjugate.

Width budget. Each entry of $\mathrm{adj}(B)$ is a signed sum of six 3×3 minors, each a product of three basis entries. For $q = 1$ (the standard order $\mathcal{O}_0$ with basis $\{1, i, (i+j)/2, (1+k)/2\}$ at denom 2), the basis entries fit in $u64$, so each minor is at most $\sim$200 bits and the full computation fits comfortably in `BigInt⟨4⟩` (256 bits). For $q \geq 5$, the iso-transformed basis matrices have entries up to $\sim 2^{250}$ (e.g. $\mathcal{O}_6$ at $q = 97$, row 1 col 3, $\sim 2^{253}$). Then:

- Each 3×3 minor reaches ~ 750 bits;
- $\mathrm{adj}(B)_{ij}$ is bounded by $6 \cdot 2^{750} \approx 2^{753}$;
- $(\mathrm{adj}(B) \cdot v)_i$ adds four products $\mathrm{adj}_{ij} \cdot v_j$ where $|v_j| \lesssim 2^{256+125} = 2^{381}$, reaching $\sim 2^{1136}$ before the final division by $\det(B) \sim 2^{750}$.

`BigInt⟨4⟩` wraps silently on overflow; the adjugate values are numerically wrong while the divisibility check $\bmod \det(B)$ may still succeed by coincidence, producing a γ whose nominal coefficients c_i do not satisfy $\gamma = \sum c_i b_i$. Two distinct symptoms:

1. RepresentInteger silently returns a γ with $\mathrm{nrd}(\gamma) \neq m$. Verifiable by computing $(a^2 + b^2 + p(c^2 + d^2))/r^2$ from the returned coordinates and comparing to the requested m.
2. `action_matrix`$(\beta, \mathcal{O}_t)$ returns `None` (the divisibility check fails for some β that mathematically lives in $\mathcal{O}_t$), aborting the outer-chain construction.

Fix. Widen both call sites to `BigInt⟨20⟩` (1280 bits) before invoking `decompose`, then narrow the resulting coefficients back to `BigInt⟨4⟩`. Width 12 (768 bits) suffices for the adjugate alone but not for $\mathrm{adj} \cdot v$ at $q \geq 41$; width 20 leaves comfortable margin. The widening is local to the decomposition and adds no asymptotic cost (decomposition runs once per γ).

Catching this earlier. We added the regression test `action_matrix_scalar_three_alternate_orders` (`deuring/tests.rs`) which calls `action_matrix`$(3 \cdot 1, \mathcal{O}_t)$ for every t and asserts the result equals $3I$. The test panics on the narrow-width path for the first $q \geq 5$ order whose adjugate intermediates overflow. An equivalent check at RepresentInteger time would compute $\sum c_i b_i$ from the returned coefficients and verify it equals γ.
Advice for implementers: Cramer's rule's intermediate sizes scale with $\binom{n}{k}$ products of basis entries; for $n = 4$ this is $4 \cdot 6 = 24$ summed triples plus a multiplication by v before division by $\det(B)$. Pick the working width to absorb $4 \cdot \log_2(\max |B_{ij}|) + \log_2(|v|)$, not just $\log_2(\max |B_{ij}|^4)$. The overflow is silent because `BigInt` wraps on multiply; test `decompose` with synthetic elements at every extremal order's full width.

5.13 Cross-order β post-processing missing in SuitableIdeals

When SuitableIdeals extends Algorithm 3.16 with the multi-order optimization (`dim2id2iso.c:534-566`), it enumerates short vectors in *seven* different lattices:

- $t = 0$: the input ideal I' (LLL-reduced equivalent of the caller's I).
- $t \geq 1$: the cross-order pushforward $\overline{I'} \cdot J_t$, where J_t is the precomputed connecting ideal to the alternate order $\mathcal{O}_t$.

A short vector β_t in lattice $t \geq 1$ is an element of $\overline{I'} \cdot J_t \subseteq B$, but downstream code in IdealToIsogeny expects $\beta_t \in I$ (the caller's original ideal) so it can compute $\theta = \beta_2 \cdot \overline{\beta_1}/\mathrm{nrd}(I)$ and apply the endomorphism via $\mathcal{O}_0$'s action matrices.

The C reference does this transport explicitly at `dim2id2iso.c:651-672`:

```
if (j1 != 0 || j2 != 0) {
    ibz_div(&delta.denom, &remain, &delta.denom, &lideal->norm);
    ibz_mul(&delta.denom, &delta.denom, &conj_ideal.norm);
}
if (j1 != 0) {
    quat_alg_mul(beta1, &delta, beta1, Bpoo);
    quat_alg_normalize(beta1);
    quat_alg_conj(beta1, beta1);
}
// (analogous for beta2)
```

where `delta` is the LLL-first vector δ of I' conjugated, with denominator adjusted to $d_{I'} \cdot \mathrm{nrd}(\overline{I'})$. After this transformation, $\beta_i \in I' \subseteq \mathcal{O}_0$. The post-condition asserted at `dim2id2iso.c:690` is `quat_lattice_contains`(I', β_i).

Our `try_find_uv` returned `factor_i.beta = sv_i.elem` directly, with no transport. For 99 of the 100 NIST-I lvl1 KATs the cross-order branch is never taken ($s = t = 0$, both $j_i = 0$), so the missing transport is a no-op. KAT 29 forces $(s, t) = (0, 1)$, and the untransported $\beta_2 \in \overline{I'} \cdot J_1$ then fails the downstream `action_matrix` call (which decomposes on $\mathcal{O}_1$).

Fix. Extend `conj_reduced_state` to carry δ (specifically: its conjugate `conj_delta`, with the original denominator $d_{I'} \cdot \mathrm{nrd}(I')$) and $d_{I'}$. After `try_find_uv` returns a successful (β_s, β_t) pair in `suitable_ideals`, construct

$$\delta_{\mathrm{pp}} \;=\; \overline{\delta} \,\Big/\, \big(d_{I'} \cdot \mathrm{nrd}(\overline{I'})\big),$$

and for each $j_i \neq 0$ replace

$$\beta_i \;\longleftarrow\; \overline{\delta_{\mathrm{pp}} \cdot \beta_i} \quad \text{(quaternion product, then normalize, then conjugate).}$$

Because δ_{pp} has wide coordinates (the LLL-first vector at the commitment ideal's working width), the intermediate quaternion product runs at `Element⟨60⟩` (matching the `conj_reduced_state` working width `W2 = 60`); the final result narrows to `Element⟨4⟩` for return.

Catching this earlier. An assertion

$$\beta_i \in I' \quad \text{(via } \texttt{lattice.contains(beta\_i)}\text{)}$$

on every SuitableIdeals return value would have fired the first time the cross-order branch was exercised. This is the same assertion the C reference makes at `dim2id2iso.c:690`.
Advice for implementers: Multi-order short-vector search enumerates in different lattices than the caller's ideal. The transport $\beta \to \overline{\delta \cdot \beta}$ is invisible at the algorithm-box level (Algorithm 3.16 says only "return β_i") but is required for downstream callers that operate on the original ideal. Make this part of the SuitableIdeals contract, not the `find_uv_from_lists` caller's responsibility.

5.14 Action-matrix order index for cross-order β

After the cross-order β transport (§5.13), β_i lives in $I' \subseteq \mathcal{O}_0$, regardless of which alternate order $\mathcal{O}_t$ the original short-vector lattice came from. The downstream IdealToIsogeny construction needs the action matrix of β_i on a torsion basis of an elliptic curve.

Two equivalent conventions. The action matrix M_β depends on a choice of *which* extremal order's generators to decompose β on:

- Decompose $\beta = \sum c_i b_i^{(s)}$ on $\mathcal{O}_s$'s basis $\{b_0^{(s)}, \ldots, b_3^{(s)}\}$ and combine with `ACTION_MATRICES[s]` (the precomputed action of $\mathcal{O}_s$'s generators on $E_s[2^f]$). This requires $\beta \in \mathcal{O}_s$.
- Decompose β on $\mathcal{O}_0$'s basis and combine with `ACTION_MATRICES[0]`. This requires $\beta \in \mathcal{O}_0$.

Both conventions yield mathematically equivalent images on the curve when the corresponding membership condition holds.

The C reference's `endomorphism_application_even_basis` (`id2iso/ref/lvlx/id2iso.c:140`) takes an explicit `index_alternate_curve` argument selecting which order to decompose on. Inside IdealToIsogeny (`dim2id2iso.c:1054, 1156`) the C reference *always* passes `0` for this argument:

```
endomorphism_application_even_basis(
    &bas2, 0, &Fv_codomain.E1, &theta, TORSION_EVEN_POWER);
```

That is, both special-order ($s = t = 0$) and cross-order ($s \neq 0 \vee t \neq 0$) flows decompose on $\mathcal{O}_0$. This is consistent with the post-transport invariant $\beta_i \in \mathcal{O}_0$.

What we did wrong. Our `to_isogeny` selected the action-matrix table by the order's q-index:

```
let s_index = EXTREMAL_ORDERS.iter()
    .position(|o| o.q() == sui.factor1.order.q())?;
let gen_matrices_s = [
    ACTION_MATRICES[s_index][3],
    ACTION_MATRICES[s_index][4],
    ACTION_MATRICES[s_index][5],
];
let m_beta1 = action_matrix(
```

```
    &sui.factor1.beta,
    sui.factor1.order.order(),   // = O_s
    &gen_matrices_s,
    f,
)?;
```

For $s = 0$ this is fine. For $s \neq 0$, after the cross-order transport $\beta_1 \in \mathcal{O}_0$, not $\mathcal{O}_s$, so `action_matrix`$(\beta_1, \mathcal{O}_s, \ldots)$'s `decompose` step rejects β_1 as "not in the lattice" and the whole IdealToIsogeny call returns `None`.

Fix. Gate the order and gen-matrices selection on `cross_order` $= (s \neq 0 \vee t \neq 0)$, picking $\mathcal{O}_0$ and `ACTION_MATRICES[0]` when set:

```
let cross_order = s_index != 0 || t_index != 0;
let order_for_decompose_s = if cross_order {
    EXTREMAL_ORDERS[0].order()
} else {
    sui.factor1.order.order()
};
let gen_matrices_s = if cross_order {
    [ACTION_MATRICES[0][3], ACTION_MATRICES[0][4], ACTION_MATRICES
        [0][5]]
} else {
    [ACTION_MATRICES[s_index][3], ACTION_MATRICES[s_index][4],
        ACTION_MATRICES[s_index][5]]
};
// (analogous for t)
```

Catching this earlier. Two checks would have flagged this:

- Same membership assertion as §5.13 (`lattice.contains(beta_i)` for the order being decomposed on).
- Determinant cross-check: $\det(M_\beta) \pmod{2^f} \stackrel{?}{=} \mathrm{nrd}(\beta) \pmod{2^f}$ on every `action_matrix` return. When this fails, the decomposition is on the wrong order or has overflowed.

Advice for implementers: When a multi-order optimization produces β in a transported form, the downstream action-matrix call should specify "decompose on the order β now lives in," not "decompose on the order β originated from." The action of an endomorphism on a curve does not depend on which integral maximal order we choose to expand it in, but `decompose`-then-recombine *does*: the recombination matrices must come from the same maximal order the decomposition is taken in.

5.15 Unbounded x-coordinate search in TorsionBasisFromHint fallback

TorsionBasisFromHint (Algorithm 2.2) reconstructs a 2-torsion basis from a curve coefficient A and a 7-bit hint. The hint h encodes the smallest integer n such that either nA (when A is a non-residue) or $-A/(1+ih)$ (when A is a residue) is a valid x-coordinate on E_A. Honest signing always finds an admissible n in the first few values, so the 7-bit hint suffices. The spec also describes a fallback path for $h = 0$: search from $n = 128$ upward until the predicate is satisfied.

The spec's pseudocode (Algorithm 2.2) writes the fallback as *"increment n until the predicate holds."* Read literally, that is an unbounded search. In `verify`, the function is called on three curves (E_{pk}, E_{aux}, E_{chl}) all reconstructed from signature bytes, so the

curve coefficient A is attacker-controlled. An adversary can supply a malformed signature whose A never satisfies the on-curve / non-residue predicate inside the $\mathbb{F}_{p^2}$ search window, and verify spins forever.

The Wycheproof suite for SQIsign includes a regression vector for exactly this case (`tcId = 49` in the verify file, flag `HintOverflow`, comment *"fuzz crash: integer overflow in find_na_x_coord"*). An implementation that translates the spec's pseudocode directly into a `while`-loop hangs on that vector regardless of language.

Fix. Bound the search by a constant well above anything observed on honest signatures (the largest n in practice is below 128, so a cap of 2^{16} leaves five orders of magnitude of headroom) and make TorsionBasisFromHint fallible. In `verify`, treat `None` as a verification failure; on the signing side the same call is on the signer's own curve, where the search always succeeds, so a non-recoverable assertion is appropriate. The fix is independent of language or counter width — a fixed-width counter that wraps silently (`u8` in our case, but the same risk exists in any C implementation using `uint8_t` or `int`) only amplifies the same algorithmic gap: a search whose termination proof relies on *"honest input always works"* is not a search at all when applied to attacker-controlled data.

After bounding the search and propagating the failure, the Wycheproof verify suite passes in ~ 0.5 seconds (49 of 49 vectors).

Advice for implementers: Every loop in `verify` that depends on attacker-controlled data needs a hard iteration cap. Search loops with implicit termination ("increment until the predicate holds") are the canonical hazard: a loop whose termination proof relies on *well-formed input* is unbounded on malicious input. Minimum hygiene: cap the iteration count, return an optional, and propagate to a verify-time rejection. Spec authors should state the cap explicitly (see §?? of the spec review). The Wycheproof project's signature suite catches exactly this pattern with `HintOverflow`-flagged test vectors — run those before shipping any signature library.

6 Type Safety for Cryptographic Invariants

Rust's type system lets us make illegal states unrepresentable. Each of the following types prevents a class of bug that would compile silently in C.

6.1 IsogenyDegree

Isogeny degrees in SQIsign are always positive odd integers bounded by 2^{f-2} (246 bits for NIST-I). In the C reference, they are bare `ibz_t` values—the same type used for everything else. Nothing prevents passing an even number or a negative value.

We represent degrees as `IsogenyDegree([u64; 4])`—unsigned, positive by construction, oddness enforced by the only constructor:

```
pub fn new_odd(limbs: [u64; 4]) -> Option<Self> {
    if limbs[0] & 1 == 0 { return None; }
    Some(Self(limbs))
}
```

Passing an even number to `FixedDegreeIsogeny` is a type error.

6.2 TorsionExponent

Torsion exponents (e such that 2^e divides the group order) are bounded by $f = 248$. A bare `u32` allows 4 billion values; only 249 are valid. `TorsionExponent(u32)` with a range check in the constructor catches this at the API boundary. Every function that takes a

torsion exponent—`action_matrix`, `to_kernel`, `fixed_degree_isogeny`—uses this type, not `u32`.

6.3 HnfLattice vs. Lattice

Lattice containment checking requires Hermite Normal Form. Calling it on a non-HNF lattice silently produces wrong answers. We use two types: `Lattice<N>` (arbitrary basis) and `HnfLattice<N>` (guaranteed HNF, constructed only via `From<Lattice>`). The `contains()` method exists only on `HnfLattice`. You cannot call it on the wrong type.

6.4 IdealFactor

`SuitableIdeals` returns two factors, each with an element β, a degree d, and a parent order. In the C reference, these are six separate variables. Accidentally pairing one factor's degree with the other's element is a plausible mistake—the types are the same. Our `IdealFactor` struct bundles them:

```
pub struct IdealFactor {
    pub order: &'static ExtremalOrder<4>,
    pub beta: Element,
    pub degree: IsogenyDegree,
}
```

Cross-wiring requires reaching into both structs and swapping fields deliberately—not something you do by accident.

7 Constant-Time Considerations

7.1 The spec's silence on side channels

The SQIsign specification analyzes `SuitableIdeals` only in terms of failure probability (§9.3.2), not side-channel resistance. By tracing Algorithm 4.2 (signing), we confirmed that the quaternion algorithms operate on data derived from the secret key I_{sk} (lines 13–19). The variable-time behavior of the C reference is a pragmatic gap, not a deliberate security argument.

7.2 Published CT techniques

Seven papers form a complete constant-time stack for SQIsign v2:

- Kouider et al. [**?**] provide constant-time big integer arithmetic (division, exponentiation, square root) up to ∼12,000 bits.
- Hanyecz et al. [**?**] give the first constant-time LLL-like lattice reduction for dimension 4, replacing the variable-time L2 algorithm used in `SuitableIdeals`.
- Basso et al. [**?**] cover CT HNF, CT `IdealGenerator`, and CT `RepresentInteger`—the three core quaternion subroutines.
- Kim et al. [**?**] prove explicit worst-case size bounds for all quaternion intermediates in Round-2 SQIsign, enabling fixed-precision (no multi-precision) arithmetic.
- Mukherjee et al. [**?**] and Jacquemin et al. [**?**] demonstrate and propose mitigations for SPA attacks on Cornacchia's algorithm—the most urgent constant-time target.

7.3 Current status

Signing and key generation are **not yet constant-time.** The entire quaternion arithmetic layer—lattice reduction (L2/LLL), Hermite Normal Form, Cornacchia/RepresentInteger, and IdealToIsogeny—is variable-time on secret-derived data. Every such code path is annotated with `TODO(ct)` citing the Algorithm 4.2 line that makes the input secret-derived.

Field arithmetic (§**??**), curve operations, and verification are constant-time or operate on purely public data.

We plan a six-phase CT hardening immediately after achieving end-to-end interoperability.

8 Verification Strategy and Threat Model

We follow the libcrux[**?**] pattern: a portable Rust path serves as the source of truth for formal verification (via hax[**?**] extracting to F*), while a parallel `asm`-feature path provides performance. The two are kept byte-equivalent through differential testing. This section enumerates the verification scope, what each layer catches, and which residual trust assumptions remain.

8.1 Layered verification

Table 1: Verification layers and the property each provides.

Layer	Tool	What it catches
Algorithmic CT	hax + F* + `secret_integers`	Branches, indexing, division on secrets in Rust
Memory safety	F* type-check	Panics, integer overflow, OOB access
Spec equivalence	F* refinement proofs	Functional correctness vs. Algorithm 4.x
Compiled-code CT	LLVM CT-coverage runs	Compiler not reintroducing branches
Empirical CT	`dudect`, `tacet`	Wall-clock timing variance under load
Asm-vs-Rust	Differential property tests	Output equivalence on N random inputs
KAT correctness	100 NIST PQC vectors	End-to-end interop with C reference

8.2 What hax verification provides

A successful hax extraction plus F* type-check on the portable Rust path proves:

- **Panic freedom.** No path through the verified modules can `unwrap`, panic on integer overflow, index out of bounds, or trigger a `debug_assert!`.
- **Memory safety by construction.** Hax refuses to extract `unsafe`, raw pointers, FFI, or inline asm. The verified path is structurally safe.
- **No computation over secret data.** With one annotation per secret-bearing field (`Coordinate`, `BigInt` representations of secret keys, etc.), F* rejects any branch on, indexing by, or division/modulo by a secret value. Existing `subtle::Choice` / `ConditionallySelectable` usage flips from convention to proof.

The codebase's `TODO(ct)` markers (one per known variable-time path on secret data, see §7) become the exact set of locations where the `secret_integers` annotation must succeed; if any remain after CT hardening, F* extraction fails and surfaces the gap.

8.3 Limitations of hax-based verification

Hax verifies the Rust source as written; the property landscape *below* that level is not in scope. We list the limitations explicitly so users understand the residual trust:

- **Cache-timing channels** on data-cache or instruction-cache patterns that depend on secret-derived prior state. Hax models direct indexing (`arr[secret_idx]`) but not prefetcher behavior, code-cache occupancy of differently-sized callees, or set-associative cache contention with co-tenants.
- **Variable-time hardware instructions** (`DIV` on `x86_64`, `UDIV` on `aarch64`, `LZCNT` / `POPCNT` on some older parts). F* rejects *secret-dependent* division explicitly, but a non-rejected ordinary integer divide still has data-dependent latency on the underlying hardware.
- **Speculative execution** (Spectre v1/v2/v4, RIDL, ZenBleed, ...). Hax models architectural semantics, not the speculative paths the CPU explores. Mitigation is out-of-scope for the verified path; users requiring Spectre resistance must add platform-specific barriers.
- **Power and EM side channels.** Outside any compiler-level analysis.
- **Allocator timing.** A first-time `Vec::with_capacity` that triggers a syscall is a side channel hax does not see. Hot-path allocations on secret-derived sizes are addressed by code review and by structural use of fixed-width `BigInt<N>`.
- **Cross-process timing attacks.** Hyperthreading, SMT contention, scheduler-induced bandwidth. Mitigations are at the OS/sysadmin layer.
- **Asm path.** Anything inside an `asm!` block or behind `cfg(feature = "asm")` is unverified by hax. The asm path is treated as *trusted-but-tested*: differential property tests prove byte-equivalence against the (verified) portable path, but not independent CT on the asm itself.

8.4 Residual trust assumptions

A user of `sqisign-selkie` with both hax verification and the asm path enabled is trusting:

1. Correctness of `rustc`, LLVM, and the F* type checker.
2. The hax extraction toolchain as a translation between Rust and F*.
3. The C reference implementation's algorithm choices, where we follow them (e.g., dual-LLL ball sampling, the C ref's kernel construction in `compute_dim2_isogeny_challenge`).
4. Hardware implementation of the chosen targets meeting the side-channel claims (variable-time instructions absent on the asm hot path; no Spectre exploitation in the deployment context).
5. The asm path's correctness (byte-equivalent to the verified Rust path for all encountered inputs, beyond the N random inputs covered by differential tests).

8.5 Roadmap and current status

The current state of each layer. All items below are explicitly **not yet shipped** unless marked **done**; the roadmap is intentionally honest about what verification claims this codebase *does* and *does not* support today.

- KAT correctness (Layer 7): **partially done**. *Verify* accepts 100/100 C ref signatures; *sign* completes for $\geq$ 50/100 KAT seeds within the current CI per-test budget. All completed signatures cross-verify against the C ref harness. The remaining gap is sign throughput, not correctness.
- Empirical CT (Layer 5): **infrastructure present, not a verification claim**. `dudect` and `tacet` runs are wired into CI; baseline measurements exist but the codebase is not yet constant-time on secret-derived paths (see §7).
- Asm-vs-Rust differential testing (Layer 6): **NOT STARTED**. Activates when the asm path lands.
- Algorithmic CT via hax + `secret_integers` (Layer 1): **NOT STARTED**. Hax extraction is queued behind asm-Fp to avoid re-verifying an evolving impl.

- F* memory safety / panic freedom (Layer 2): **NOT STARTED**. Lands together with hax extraction.
- Spec equivalence proofs (Layer 3): **NOT STARTED**. Out of scope for the v0.1 tag; deferred indefinitely.
- Asm formal verification: **NOT STARTED**. See the open work item below.

Open work item: formal verification of the asm path (*TODO, not yet started*). Hax does not see asm. The trust gap above (item 5 in "residual trust") is the standard libcrux model: byte-equivalent diff tests against a verified Rust source. For higher-assurance deployments, a separate effort using EverCrypt/Vale-style asm verification [**?**] would close that gap with a formal proof that the asm respects the same functional + CT specification F* verifies on the Rust path. This is tracked here as a TODO for a post-tag milestone; shipping it requires writing Vale specifications for the $\mathrm{Fp}/\mathrm{Fp}^2$ hot paths and integrating an additional verification pipeline. The plan, in order, is:

1. **(in progress)** Tag spec/C-ref correctness (current milestone target).
2. **(not started, queued next)** Implement asm-Fp on `aarch64` and `x86_64`; diff-test against portable Rust.
3. **(not started)** Hax-extract the portable Rust path to F*; verify memory safety, panic freedom, and `secret_integers` CT.
4. **(not started, future)** Vale verification of the asm path against the same F* specification.

Until each step lands, the corresponding verification claim *does not apply* and should not be relied on. Users should treat this section as a forward-looking roadmap, not a statement of current properties.

8.6 What hax cannot replace

Independent of the side-channel layers above, hax is not a substitute for:

- Algorithmic correctness review. F* proves the code matches a *spec*; the spec itself can be wrong, and SQIsign's specification has historically been ambiguous in ways our bug catalog (§4) documents.
- KAT testing. F* proves correctness up to the specification's expressed properties; KAT vectors prove interoperability with deployed implementations. Both are needed.
- Negative testing. Wycheproof-style adversarial vectors catch failure modes (malleable signatures, parser confusion) that functional verification often does not.

9 Test Infrastructure

Beyond the 100 KAT vectors from the C reference implementation (all of which verify successfully), we maintain a growing suite of negative and vulnerability test vectors in the C2SP/wycheproof JSON format [**?**]. These vectors are designed to detect not just bugs but *vulnerabilities*—they exercise rejection paths that the KAT vectors, being all valid, never touch.

Invalid vectors are independently generated by byte-level perturbation (no dependency on the C reference implementation). Valid vectors cite the C reference KAT file as their provenance; we intend to replace them with independently Sage-generated vectors for eventual upstream contribution to the C2SP/wycheproof project.

9.1 Algorithm naming

We identify the parameter set as `SQIsign_248`, where 248 is the torsion exponent f in $p = 5 \cdot 2^{248} - 1$. The exponent is the fundamental parameter from which all others derive:

field size, isogeny degrees, key sizes, signature sizes. Unlike "NIST-I" (a security-level label that could be reassigned to different parameters) or "353" (a derived quantity—the secret key size in bytes—that would change if the encoding changes), $f = 248$ pins the actual cryptographic parameters.

9.2 Verification vectors

Table 2 summarizes the verification vector categories. Each vector consists of a public key, message, signature, and expected result (`valid` or `invalid`). Invalid vectors are derived from valid KAT vectors by targeted perturbation.

Table 2: Verification test vector categories for `SQIsign_248` (C2SP/wycheproof JSON format).

Category	Count	What it detects
Normal (valid KAT)	2	Baseline: C ref signatures verify.
WrongMessage	2	Different, empty, extended, and truncated messages.
WrongPublicKey	1	Cross-key verification.
BitFlip (per field)	7	Single-bit corruption in each of the seven signature fields.
AllZero / AllOnes	2	Degenerate byte patterns.
ExponentOverflow	4	$n_{\text{bt}} + r_{\text{rsp}} > e_{\text{rsp}}$, causing e'_{rsp} underflow.
NonCanonicalFp	3	Field elements $\geq p$ in `E_aux` or pk. Silently reduces mod p, producing the wrong curve.
SingularCurve	3	$A = \pm 2$ (singular Montgomery curve, discriminant zero) in `E_aux` or pk.
SpecialJInvariant	1	$A = 0$ ($j = 1728$, extra automorphisms $\pm 1, \pm i$).
StartingCurve	1	`E_aux` $= E_0$ ($A = 6$).
DegenerateMatrix	2	`M_chl` all-zero or identity.
ZeroChallenge	1	Challenge $= 0$ (degenerate Fiat–Shamir).
ScalarOverflow	1	Matrix entry $\geq 2^{e_{\text{rsp}}}$.
ChallengeOverflow	1	High bits set beyond $e_{\text{chl}} = 122$.
SplicedField	3	Signature field replaced with the corresponding field from a different valid signature.
MalformedPk	2	All-zero or all-`0xFF` public key.
Total	**39**	

Finding: verification panics on malformed input. Several invalid vectors (bit flips in `E_aux`, `M_chl`, and `chl`) reach the $(2, 2)$-isogeny splitting step (Algorithm 8.42, GetIndexSplitting), which uses a `debug_assert!` to check that exactly one null-point index is zero. Malformed signatures violate this invariant, and the assertion panics rather than returning an error. In debug builds this is a denial-of-service vector: any untrusted signature can crash the verifier. In release builds the assertion compiles out and the function returns a wrong index silently. The correct fix is to return `Option<usize>` and propagate the error as `VerificationFailed`. Our test runner uses `std::panic::catch_unwind` as a temporary workaround.

9.3 Key generation vectors

We maintain 12 keygen/sk-parsing vectors:

- Deterministic keygen from KAT seeds produces the expected (pk, sk) byte-for-byte.

- The pk embedded in the first 65 bytes of sk matches the standalone pk.
- Malformed sk rejection: truncated, extended, all-zero, all-`0xFF`, single-bit flips in each region (pk, ideal norm, M_{sk} matrix), and a "Frankenstein" sk with the pk from one vector spliced onto the sk body of another.

9.4 Signing vectors (planned)

The signing vector file is a skeleton with one vector (C ref KAT round-trip) and a roadmap for categories to add as signing stabilizes:

- **NonceIndependence**: same (sk, msg) with different randomness must produce distinct commitment curves. Nonce reuse leaks the secret key.
- **CornacchiaBranching**: inputs crafted so that RepresentInteger's Cornacchia subroutine takes different branches, targeting SPA vulnerabilities [**?**].
- **ResponseEdgeCases**: signatures with $r_{\text{rsp}} > 0$ or $n_{\text{bt}} > 0$, exercising the even-response isogeny and backtracking paths that are rare in KAT vectors.

9.5 Mutation testing

We use `cargo-mutants` [**?**] to verify that the test suite catches bugs, not just that it runs. Mutation testing inserts faults (replacing operators, return values, conditionals) and checks whether any test fails. A surviving mutant is a missing test.

Running `cargo-mutants` on the `keys/` module (212 mutants) found 8 surviving mutants—6 in `ChallengeMatrix::from_bytes` validation and 2 in keygen—all of which represented real test gaps. Writing targeted tests killed the validation mutants; the keygen mutants survive because keygen tests are slow (each invocation runs a $(2, 2)$-isogeny chain) and are marked `#[ignore]`.

CI runs incremental mutation testing on every pull request (only changed code via `--in-diff`) and a full sharded run weekly.

10 Current Status and Future Work

Verification is complete: all KAT vectors from the C reference implementation verify successfully, exercising the full pipeline from torsion basis construction through the $(2, 2)$-isogeny chain and splitting.

Signing compiles end-to-end and is structurally complete. The commitment phase, challenge derivation, and response phase are wired together, with `Order<N>` tracking maximal orders through the Deuring correspondence. Two coupled blockers remain:

- The sampling radius in the response phase is a placeholder; the correct value $D_{\text{rsp}} \cdot D_{\text{mix}}^2 \cdot 2^{f+1} \approx 2^{1399}$ requires wider lattice arithmetic ($\sim$22 limbs for the radius, $\sim$12 for lattice entries).
- Degree computations in the response phase are missing a D_{mix}^2 division, coupled with the radius fix.

Key generation (Algorithm 4.1) is not yet implemented. Signature serialization to the 148-byte wire format is implemented.

Signing and key generation are **not yet constant-time.** The quaternion arithmetic layer is variable-time on secret-derived data (see §7 for details and our hardening plan).

Next steps.

1. Widen the response-phase sampling and degree arithmetic to achieve a first successful signature;

2. Wire key generation;
3. Achieve full KAT interoperability (sign + verify round-trip);
4. Execute the six-phase CT hardening plan (§7).